Development of a Linked Open Vocabulary for a Feedback Survey

A Thesis

Submitted to the Faculty of Graduate Studies and Research

In Partial Fulfillment of the Requirements

For the Degree of

Master of Science

in Computer Science

University of Regina

By

Parita Dipakkumar Amin

Regina, Saskatchewan

2022

Abstract

The World Wide Web (WWW), as defined by the World Wide Web Consortium (W3C), is "the universe of network-accessible information, the embodiment of human knowledge"[1]. With development over the years, the global web of linked documents has become the global web of linked data, which is popularly known as the Semantic Web.

The Semantic Web is now an established topic of research but there remain many unanswered questions. Linked Open Data (LOD), as the highest standard for linked data, is a worthy goal for researchers and data providers who seek to contribute to the development of the Semantic Web. Data that are linked and open facilitate discovery and reuse. Likewise, vocabularies used to describe the semantic relationships amongst data that are linked and open facilitate discovery and reuse. Yet, it is not always a straightforward matter to take data that may be somehow accessible on a webpage written in Hyper Text Markup Language (HTML) and publish it as LOD. Many complimentary technologies are used in handling LOD on the web, particularly the Resource Description Framework (RDF), the Web Ontology Language (OWL), and the Simple Protocol and RDF Query Language (SPARQL).

last paragraph news some attention – presently commented out

[1] https://www.w3.org/WWW/

Acknowledgements

I would like to take an opportunity to thank all the people who have supported me during my research period in some or the other way.

I would like to express my soulful gratitude to my supervisor Dr. Daryl H. Hepting for his valuable time, guidance, suggestions and comments throughout the course of the research. He was always open to all my questions and queries related to my research work as well as while writing this thesis. He provided me with the space required to make this study as my own work, at the same time he made sure that I am leading on the right path by giving constant feedback and suggestions whenever I was lost or stuck. I would also like to thank the Department of Computer Science and Faculty of Graduate Studies and Research for their academic as well as administrative support as and when required.

Lastly, I am grateful to my parents for always being there for me regardless of the distance and time-difference between the two different parts of the world. This would have been next to impossible for me if not for them. I would like to specially thank my brother Shivam for being the family here and for encouraging and motivating me during the times I was low. My special thanks to all my friends for all their support.

Post-Defense Acknowledgements

Table of Contents

List of Figures

List of Tables

List of Abbreviations

API	Application Programming Interface p. 27
CERN	Conseil Européen pour la Recherche Nucléaire p. 1
CSV	Comma Separated Value p. 26
DL	Description Logics p. 49
ER	Entity Relationship p. 10
GIF	Graphics Interchange Format p. 76
GREL	Google Refine Expression Language p. 26
GUI	Graphical User Interface p. 26
HTML	Hyper Text Markup Language p. i
HTTP	Hyper Text Transfer Protocol p. 2
HTTPS	Hypertext Transfer Protocol Secure p. 2
IDE	Integrated Development Environment p. 48
IRI	Internationalized Resource Identifier p. 42
ISO	International Organization for Standardization p. 40
JPEG	Joint Photographic Experts Group p. 76
JSON	JavaScript Object Notation p. 50
LOD	Linked Open Data p. i
N3	Notation3 p. 12
OBDA	Ontology-Based Data Access p. 49
OWL	Web Ontology Language p. i
PNG	Portable Network Graphics p. 76
RDF	Resource Description Framework p. i
RDFS	Resource Description Framework Schema p. 13
RML	RDF Mapping Language p. 26
SHACL	SHApes Constraint Language p. 49
SPARQL	Simple Protocol and RDF Query Language p. i
SVG	Scalable Vector Graphics p. 77
SWT	Semantic Web Technologies p. 49
TSV	Tab Separated Value p. 50
Turtle	Terse RDF Triple Language p. 12
URI	Uniform Resource Identifier p. 2
URL	Uniform Resource Locator p. 19
VOWL	Visual Web Ontology Language p. 76
W3C	World Wide Web Consortium p. i
WWW	World Wide Web p. i
XML	eXtensible Markup Language p. 6

Chapter 1

Introduction

Tim Berners-Lee is the father of the WWW. He wrote *Information Management: A Proposal*[1] in 1989 and 1990 to present the need for a "global hypertext system" at the Conseil Européen pour la Recherche Nucléaire (CERN). In his conclusion to that document, he wrote:

> "we should work toward a universal linked information system, in which generality and portability are more important than fancy graphics techniques and complex extra facilities. The aim would be to allow a place to be found for any information or reference which one felt was important and a way of finding it afterward. The result should be sufficiently attractive to use, that is, the information contained would grow past a critical threshold so that the usefulness of the scheme would, in turn, encourage its increased use."

In the more than 30 years that have followed that initial proposal, a great deal of progress towards Berners-Lee's vision has been realized. The original "web of documents" is becoming the "web of data" or the "semantic web". Is this next web sufficiently attractive to use? This thesis will examine this question in the context of an example.

[1] `https://www.w3.org/History/1989/proposal.html`, dated March 1989, May 1990

1.1 Motivation

In his teaching practise, my supervisor collects feedback from students in his classes each semester and makes the responses available on his website[2]. Typically, student responses were collected on paper and then entered manually into computer-readable form. Data collection since the start of the COVID-19 pandemic has changed.

Although it is valuable to have this data available on the web, it is not published as LOD. The motivation for the research presented here was to examine the process by which existing data can be published as LOD and to consider whether that process is "sufficiently attractive to use".

The published linked (structured) data which is available on the web can be efficiently retrieved and reused as, according to Ngomo et al. [36], linked data is "a set of best practices for publishing and connecting structured data on the web". After the introduction of the semantic web and the four principles of linked data for publishing and connecting structured datasets on the web by Berners-Lee, the web of documents has been gradually moving towards the development of the web of data. With time, these principles have been accepted and considered as the standard to publish structured data on the semantic web [14, 16, 17, 28]. The four principles of linked data by Berners-Lee [8] are as follows:

- Use Uniform Resource Identifier (URI) as names for things
- Use Hyper Text Transfer Protocol (HTTP)[3] URIs to look up those names
- Provide useful information, using the standards (RDF[4], SPARQL)
- Include links to other URIs to discover more things

[2] `http://www2.cs.uregina.ca/~hepting/teaching/evaluation.html`
[3] Both HTTP and Hypertext Transfer Protocol Secure (HTTPS)
[4] includes all the standards belonging to RDF family

After the development of Linked Data for the semantic web, Berners-Lee also contributed to web engineering by introducing a 5-star rating[5] system for the rating of linked data implementations. In this system, using the linked data principles, the publishers of the data on the web link their data to the existing linked data to provide reference and make it available for lookup. The data on the web, according to the 5-star system, is accepted to be 5-star data if the data:

- is structured (machine-readable)
- has open-license for access
- is described and linked to other data using URIs

A query[6] is used to retrieve information from a database that consists of multiple tables.

According to the W3C recommendation, a semantic (web) query depicts the techniques and protocols that are "used to retrieve information from the web of data" using computer programs. Web (semantic) queries use the semantics and the constitution of data on the web to perform data retrieval operations. Various semantic web technologies have been developed for data retrieval on the web. SPARQL is one of the semantic web technologies used for web queries, where it gathers all the data from different domains and treats all those datasets as local.

[5]https://www.w3.org/2011/gld/wiki/5_Star_Linked_Data
[6]https://en.wikipedia.org/wiki/SPARQL

1.2 Contribution

1.3 Organization

Following this introduction, the rest of the thesis is organized as follows. Chapter 2... Chapter 3... Chapter 4... Finally, Chapter 5 presents some conclusions and opportunities for future work.

Chapter 2

Related Work

This Chapter provides the theoretical and conceptual information that is the foundation for this thesis, focusing on the semantic web and linked open data from the perspectives of technology and usability.

2.1 The Web

The idea of the Web, as described by Berners-Lee [10], came from the positive experience of a small "home-brew" personal hypertext system. The web was first designed by Berners-Lee in 1989[1] while working at the CERN.

2.1.1 Semantic Web

The web pages for the semantic web can be written using HTML and RDF as it is the web of data, contrary to the web of documents, where the web pages were written using HTML and the links were described using hyperlinks. In the web of data, arbitrary things can be expressed using links to represent the relationship between two entities, as shown in Figure 2.1.

The latest generation of web that is still at its research and development stage is Web

[1]https://www.w3.org/History/1989/proposal.html

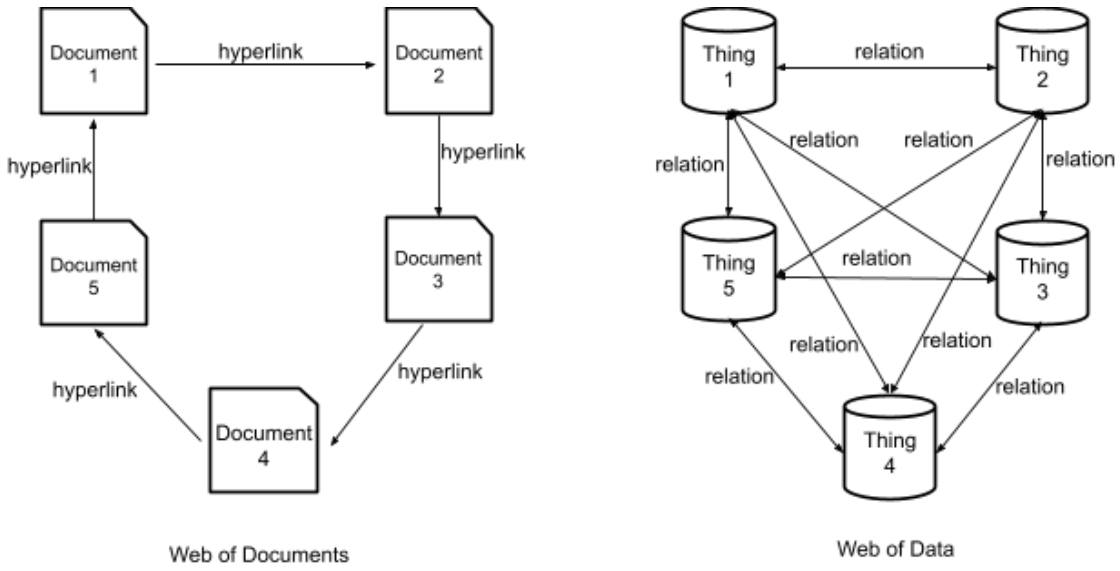

Figure 2.1: The Web to Semantic Web, according to Aghaei et al. [1]

4.0 which is supposed to be read-write-execute with the property of concurrency [19]. This version of the web is intended to work with artificial intelligence to give users an exceptional quality of experience. While describing how Web 4.0 would be, Berners-Lee [13] stated "if HTML and the Web made all the online documents look like one huge book, RDF, schema, and inference languages will make all the data in the world look like one huge database".

2.1.2 The Semantic Web Stack

The Semantic Web Stack [2] is also known as Semantic Web Cake or Semantic Web Layer Cake. The stack represents the architecture of semantic web. Figure 2.2[3] shows how semantic web principles are implemented in the layers of technologies.

The layers in the stack consist of different technologies. Going from the lowest level to the top level, the technologies are Unicode and URI; eXtensible Markup Language (XML), NS and xmlschema; RDF and rdfschema; Ontology vocabulary; Logic; Proof;

[2]https://en.wikipedia.org/wiki/Semantic_Web_Stack
[3]https://www.w3.org/2001/12/semweb-fin/w3csw

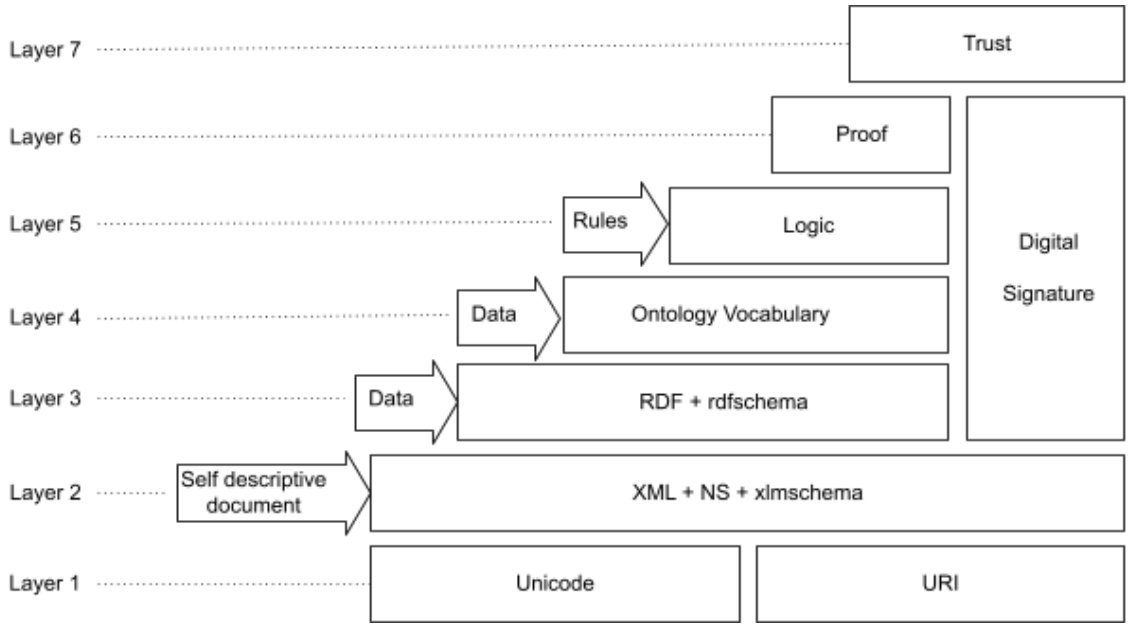

Figure 2.2: Semantic Web Stack

and Trust. Digital signatures are used as security at different levels. The layers belong to three different groups, namely, self-descriptive document, data and rules.

2.2 Linked Open Data

According to Berners-Lee et al. [1], "Semantic web is not limited to publication of data on the web; it is about making links to connect related data." The concept of Linked Open Data[4] is an amalgamation of two technologies, Linked Data and Open Data. GraphDB[5] by Ontotext, an RDF database, is an example of LOD. GraphDB promotes knowledge-discovery and efficient data-driven analytics by linking large datasets from distinct sources to Open Data.

[4]https://www.ontotext.com/knowledgehub/fundamentals/linked-data-linked-open-data/
[5]https://www.ontotext.com/knowledgehub/fundamentals/linked-data-linked-open-data/

2.2.1 Linked Data

The relationships shared among the data from different sources on the web are equally important as the access to the data for efficient working of the semantic web. The technique used to manage these datasets and their relationships is referred to as Linked Data. In other words, Linked Data[6] can be termed as a set of various techniques of publishing structured data on the web. The W3C standards and various semantic web technologies handle the distribution and engagement of the structured data across the web to aggregate the data available on the web into one huge database [13].

In 2006, Berners-Lee [15] framed four basic rules (mentioned in section 1.1) to publish and connect the structured datasets across the web. This set of rules was termed as Linked Data principles. A brief description of these principles is as follows:

1. Use URIs as names for things:

   The web identifiers, the (URIs) should be used to give unique names to all the entities (or things) on the web. URI of an entity can be used to understand that the entity in this dataset is the same as the other entity in a disparate dataset.

2. Use HTTP[7] URIs to look up for those names:

   The HTTP/HTTPS protocols allow easy retrieval of resources. Hence, along with URIs, either of the protocols can be used to search things (entities) easily. This promotes the publication of data on the web and its addition to the global data space.

3. Provide useful information, using the standards (RDF[8], SPARQL) to look up for a URI:

[6]https://www.w3.org/wiki/LinkedData
[7]Both HTTP and HTTPS
[8]includes all the standards belonging to RDF family

When an entity is searched using the URI associated with it, it is beneficial to use standards like RDF and SPARQL to query the datasets. RDF is a framework used for graphical representation of publication and distribution of data on the web, as described in detail in section 2.3. On the contrary, SPARQL is a query language used for retrieval and manipulation of data (in RDF format) on the web. It also helps with the information on relationships related to the data being searched.

4. Include links to other URIs to discover more thing:

   In the semantic web, links between two entity defines the relationship using the basic entity-relationship model. Using links for URIs of the entities promotes interconnection of the data and search of things on the web. Connecting new data to already existing entities increases the reuse and interlinking within the domain and creates a computer-understandable network.

Publishing data on the web as per linked data principles can benefit the data providers by allowing them to use one global space to add all their data. An RDF data model and URIs are used by the linked data to name the things (entities). The linked datasets publish and locate the class-object data which in turn is accessed using the HTTP/HTTPS and maintains the connection among them [5].

A wide network of structured data, from a variety of domains, on the web has been formed obeying the principles of linked data by Berners-Lee. This has resulted in the formation of a huge database of structured linked data on the web which has further led to the concept of the LOD Cloud. The LOD cloud is a set of publicly [9] accessible/available linked datasets.

2.2.2 Linked (Open) Data

According to a handbook[9] by the Open Knowledge Foundation, "Open Data is data that can be freely used, re-used and redistributed by anyone - subject only, at most, to the requirement to attribute and share-alike". To make open data available to everyone on the web, the data don't need to be interlinked. Important features of open data are:

- Availability and access: the data should be available on the web. The users should be able to access the data easily by downloading them over the internet. It should be easy to access and modify the data.
- Re-use and redistribution: the data on the web must have permissions to re-use and redistribute in combination to other datasets.
- Universal participation: the data should be available to use, re-use, and redistribute for all the users without any discrimination to an individual or a group.

2.3 Resource Description Framework

The RDF, initially designed as a metadata data model, is a computer-understandable specification written in a particular format. According to a proposal by W3C[10], RDF plays an important role in their "Semantic Web Vision". RDF describes an entity (or resources) and the relationships among them on the web using basic data models like the Entity Relationship (ER) model.

RDF is a W3C[11] specification for describing resources on the web. RDF uses "properties" and "property values" to describe the resource which is identified by web identifiers called URI (section 2.3.1). There are three types of objects in RDF, namely, resources,

[9]https://opendatahandbook.org/guide/en/what-is-open-data/
[10]https://www.w3.org/TR/PR-rdf-syntax/Overview.html
[11]https://www.w3schools.com/xml/xml_rdf.asp

properties, and statements. Each data object in a dataset on the web is considered a resource and is named using a URI with an optional ID. To search for the resources on the web, the resources must be named using the HTTP/HTTPS URIs. The W3C has defined the "rdf:type" property to represent the "object of one or more classes" [21]. Resources are described by their characteristics, called "property". The object of RDF that is used to combine a resource with a property of its value is a "statement". A statement, in general terms, can be seen as building blocks of a statement in English, which are a subject, an object, and a predicate. An object in a statement can be either a property value, a resource, or a literal. RDF, according to Swick and Lassila [6], can be represented in three ways:

1. Using XML: an RDF/XML document
2. As triples: subject-predicate-object form
3. In graphical form

Figure 2.3 shows an example of an RDF/XML[12] document. The resource being described is "https://www.semanticweb.org/pda324/ontologies/survey" and "Survey Details" is the property value. The document consists of various XML tags enclosed between the angular brackets ("<" and ">"). An RDF/XML document starts and ends with <rdf : RDF> and </rdf : RDF> respectively. <rdf : Description> consists of the elements that are responsible for the creation of the statement. Description tag holds an identification for the resource.

Figure 2.4 shows an example of an RDF as a triple. An RDF triple, generally, follows subject-predicate-object format. For the illustration in the figure, the subject is "https://www.semanticweb.org/pda324/ontologies/survey", predicate is "includes" and the object is "Survey Details".

[12]RDF written using XML

```
<rdf : RDF>
  <rdf:Description rdf:about="https://www.semanticweb.org/pda324/ontologies/survey">
    <s:istypeof>Survey Details</s:istypeof>
  </rdf:Description>
</rdf : RDF>
```

Figure 2.3: An Illustration of Code in an RDF/XML Document

Subject : https://www.semanticweb.org/pda324/ontologies/survey
Predicate : includes
Object : Survey Details

Figure 2.4: An Example of an RDF Triple

Figure 2.5 show an RDF in graphical format. Resource is indicated using an oval shape and a rectangle is used to represent a property value. A connector (an arrow) is used to represent a property to show that property (predicate) connects the resource (subject) with the value (object).

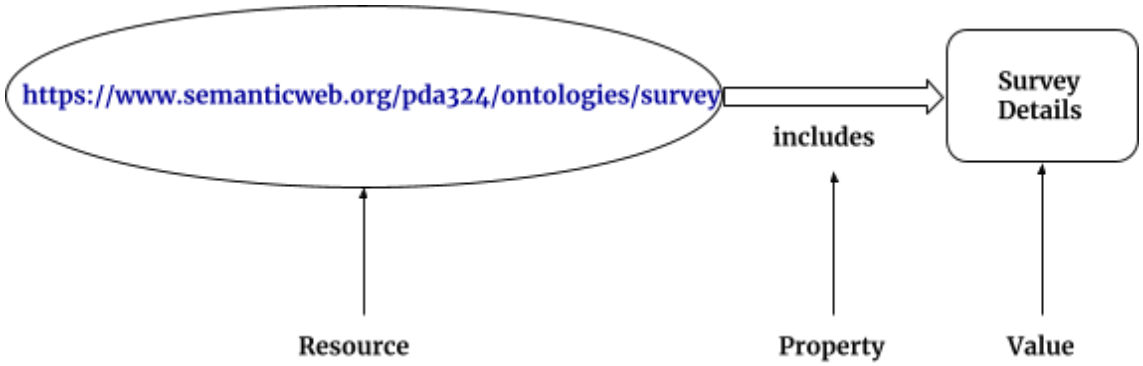

Figure 2.5: Graphical Representation of an RDF

The subject and object of the triples of RDF act as nodes in the graph. In addition to RDF/XML documentation, the RDF data model has a variety of serialization formats such as N-triples, which is a line base format where an RDF triple is used to represent each line [7]; Terse RDF Triple Language (Turtle); Notation3 (N3), which is a subset of

Turtle and N-Quads, which is an extension of N-Triples (line based) and adds a graph label to each line that allows multiple graph encoding [20].

Class/Property	Description
rdfs:Resource	All things described by RDF are called resources
rdfs:Class	The class of resources that are RDF classes
rdf:Property	The class of RDF properties. It is an instance of rdfs:Class
rdfs:Literal	The class of literal values like integers and strings
rdfs:Datatype	The class of datatypes. All the objects of this class correspond to the RDF model of datatype
rdf:HTML	The class of HTML literal values and subclass of rdfs:Literal
rdfs:range	An instance of rdf:Property that is used to state that the values of a property are instances of one or more classes
rdfs:domain	An instance of rdf:Property that is used to state that any resource that has a given property is an instance of one or more classes.
rdf:type	An instance of rdf:Property that is used to state that a resource is an instance of a class
rdfs:subClassOf	An instance of rdf:Property that is used to state that all the instances of one class are instances of another
rdfs:label	an instance of rdf:Property that may be used to provide a human-readable version of a resource's name
rdfs:subPropertyOf	An instance of rdf:Property that is used to state that all resources related by one property are also related by another

Table 2.1: Commonly used Classes and Properties of RDFS

Resource Description Framework Schema (RDFS), according to McBride [35], was introduced by the W3C as a technique to define resource types and property names. The data publishers can use RDFS to define various vocabularies in an RDF model. Some of the commonly used RDFS classes and properties[13] along with their description are listed in Table 2.1 [35].

Figure 2.6 shows a graph for the representation of RDFS for a comparatively complex and abstract example based on RDF Primer [34]. The graph has been drawn for the following statement:

[13]https://www.w3.org/TR/rdf-schema/

Romeo and Juliet is written by William Shakespeare.

Figure 2.6 is the graphical representation of an RDF with schema as well as data. In the graph, the rdf:type of "Romeo and Juliet" is "Play" which in turn is an rdf:subClassOf "Literature". Also, the rdf:type of "William Shakespeare" is "Person" which, further, is an rdfs:range of "is written by" of rdfs:domain "Literature".

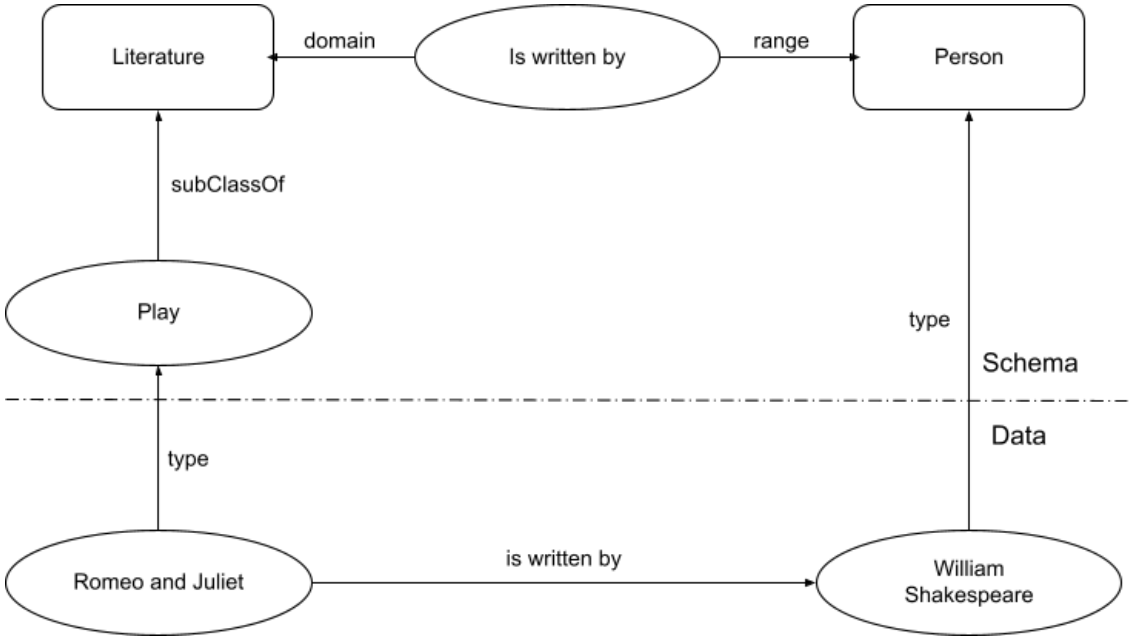

Figure 2.6: Graphical Representation of an RDFS for a Comparatively Complex Illustration, after McBride [35]

The actual RDF looks quite different from the previous example (Figures 2.3, 2.4, 2.5 and 2.6). For the representation of an actual RDF, consider Figure 2.7 for the corresponding English statement. "http://www.semanticweb.org/survey.html" is a subject URI with respondent and Robert James having URIs as "http://xyz.org/1/respondent" and "http://www.semanticweb.org/responseID/1234" respectively. The example represents a survey system which has responses as well as respondents and related details as a part of the environment.

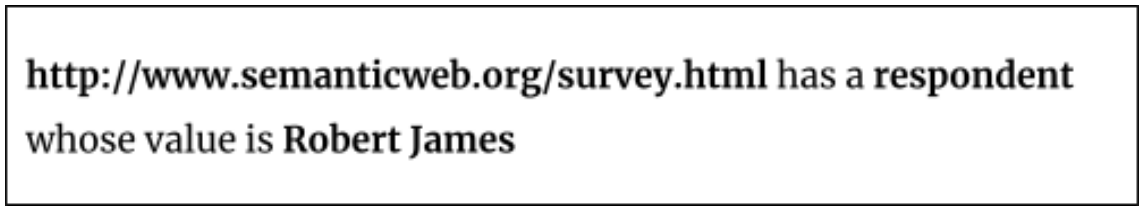

Figure 2.7: English Statement for an Example of Actual RDF

In the form of RDF triple, the statement in Figure 2.7 can be represented as shown in Figure 2.8. URI is assigned to the subject survey.html and to the object which is the response with responseID 1234 which corresponds to the property value Robert James. A URI is also assigned to the predicate (respondent).

Figure 2.8: RDF Triple for English Statement in Figure 2.7, after RDF Primer [34]

Figure 2.9 shows a graphical representation of the RDF triple in Figure 2.8. The subject and the object are two entities and the predicate being the third entity acts as a connector between them. Considering an entity-relationship data model, the predicate can be treated as a relationship shared by the subject and the object. There can be more details added to the system, that is, more statements can be added which are in relation to the one in Figure 2.7. Suppose the new statement added is:

http://www.semanticweb.org/survey.html has a *date* with value *Sept. 08, 2020*

and the URI for the date is "http://www.semanticweb.org/respDetails/date".

The new graph would be as shown in Figure 2.10 where as the set of statements in Table 2.2 shows the RDF triple notation for Figure 2.10. The first notation represents the entities, survey.html (subject) and Robert James (object), along with respondent

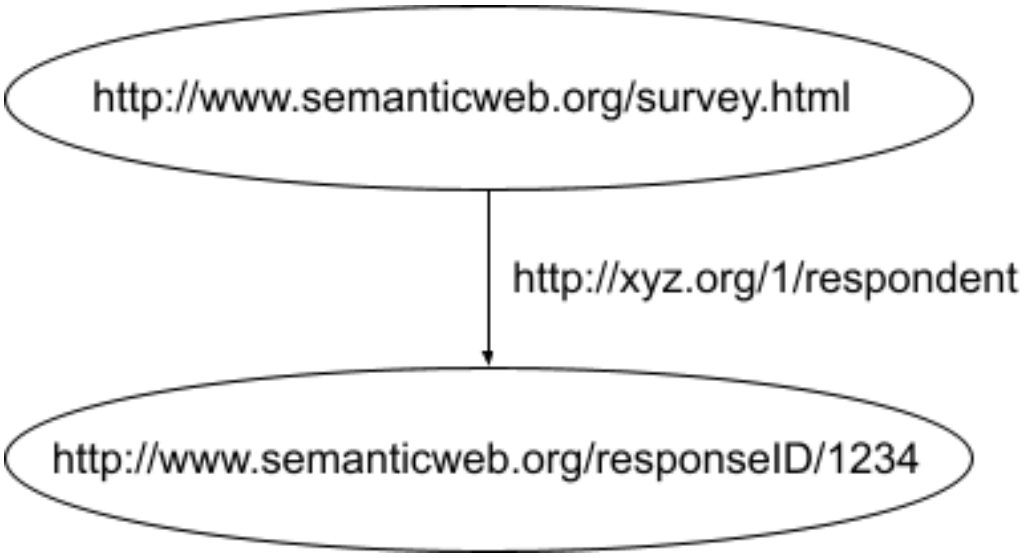

Figure 2.9: RDF Graph for English Statement in Figure 2.7, after RDF Primer [34]

(predicate) as the relation between them, using the URIs. The second notation is for the date, where instead of the URI, the value of the property, Sept. 08, 2020, is used. Here, survey.html is the subject and Sept. 08, 2020 is the object with date acting as the predicate.

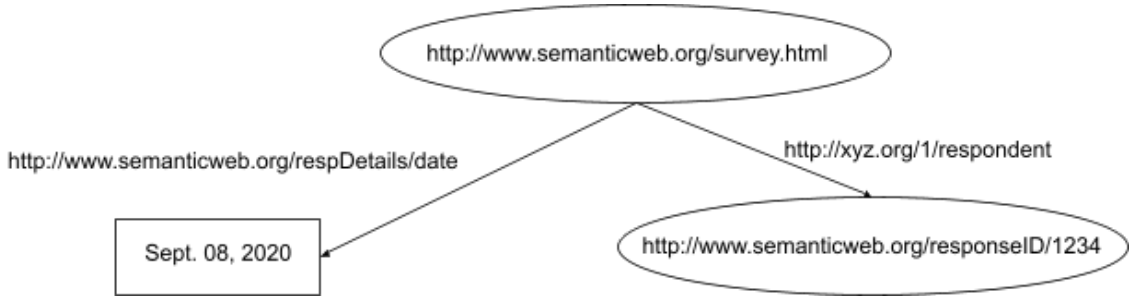

Figure 2.10: RDF Graph for Two Relative Statements, after RDF Primer [34]

<http://www.semanticweb.org/survey.html> <http://xyz.org/1/respondent> <http://www.semanticweb.org/responseID/1234> .
<http://www.semanticweb.org/respDetails/date> “Sept. 08, 2020” .

Table 2.2: RDF Notations for Figure 2.10, after RDF Primer [34]

Consider a more complex system, where the details of respondent are collected. Sup-

pose, one of the details is name and the other is address of the respondent. Figure 2.11 shows a graphical representation of this RDF system.

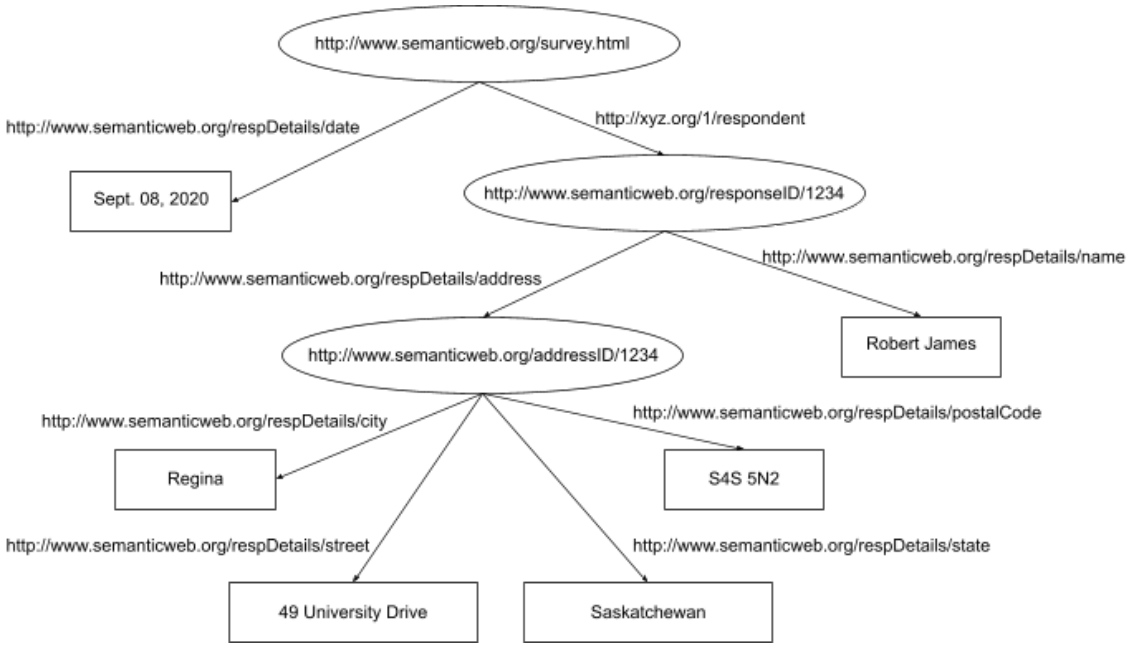

Figure 2.11: Graphical Representation of RDF with Details of the Respondent of the Survey, after RDF Primer [34]

For name, the respondent is subject and Robert James acts as object whereas respName is the predicate. The URI for respName is "http://www.semanticweb.org/respDetails/name" and the value, Robert James, is the object. On the other hand, respondent (subject) is connected to address (object) node, with URI "http://www.semanticweb.org/addressID/1234", through respAddress (predicate) with URI "http://www.semanticweb.org/respDetails/address". Further, the address node has multiple child nodes, namely, street, city, state and postal code. The address (subject) URI is connected to the values of four different object values, 49 University Drive (street), Regina (city), Saskatchewan (state) and S4S 5N2 (postalCode) where the URIs for the predicates are "http://www.semanticweb.org/respDetails/street", "http://www.semanticweb.org/respDetails/city", "http://www.semanticweb.org/respDetails/state" and "http://www.semanticweb.org/respDetails/postalCode" respectively. Instead of using an unnecessary

URI for the address entity, a blank node can be used to denote the entity, as shown in Figure 2.12. As the relationships of the node with other nodes give sufficient details without hindering the flow of the data tree, the node does not require a URIreference[14].

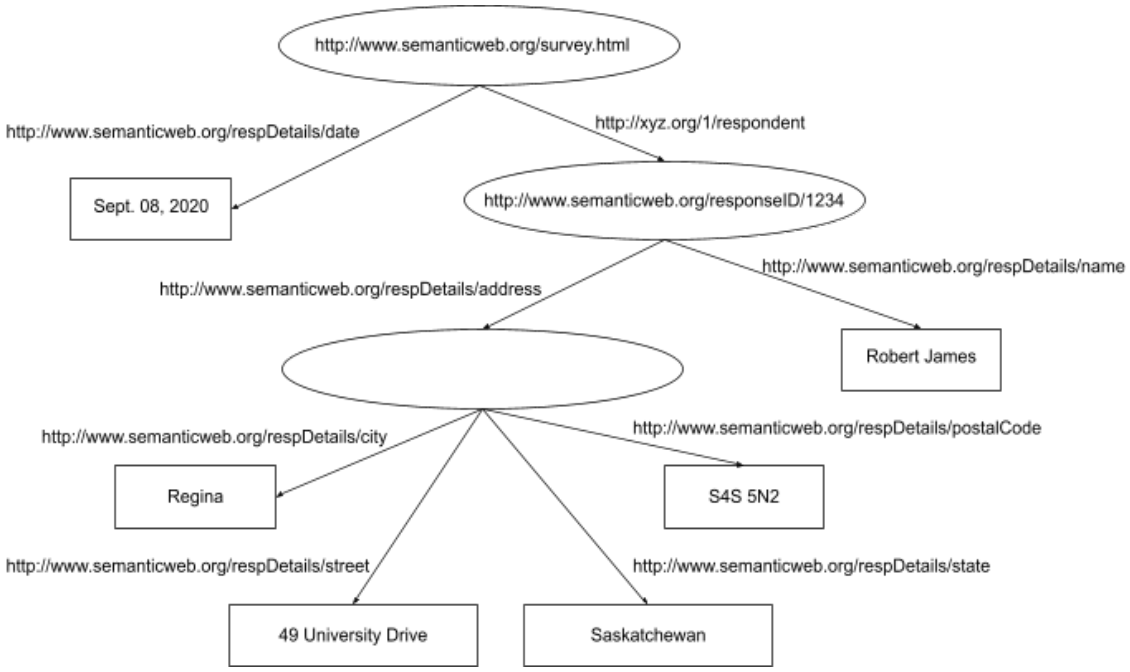

Figure 2.12: RDF Representation in Figure 2.11 without a URI (blank node) for Address Entity, after RDF Primer [34]

In 2004, W3C recommended intended goals that are to be achieved whenever an RDF is to be designed. According to the W3CRecommendation[15], the design should "have a simple data model; formal semantics and provable inference; should use an extensible URI-based vocabulary and an XML-based syntax; should support the use of XML schema datatypes; and should allow any one to make statements about any resource".

[14]https://lists.w3.org/Archives/Public/uri/2007Jul/0027.html
[15]https://www.w3.org/TR/rdf-concepts/

2.3.1 Uniform Resource Identifier

An abstract (a logical) or a physical resource on the web is identified by a unique set of ASCII characters which is referred to as a URI[16]. According to Berners-Lee et. al. [12] in RFC 3986, the unique string, on the web, follows a set of guidelines and security considerations while defining a syntax for the generic URI and determining a process for URI references' resolution. As mentioned by W3C, for the operating systems on URIs, some specific "protocols are defined depending on the UriScheme[17]". A system can perform multiple operations like "access", "update", "replace" and "find attributes" on the identified resources. In RFC 2396, Berners-Lee et al. [11] characterized URI using three definitions, as shown in Table 2.3.

2.3.1.1 Syntax of a URI

According to Berners-Lee et al. [12], the generic syntax of a URI comprises five components in a hierarchical sequence which are as follows:

URI = scheme: [//authority] path [?query] [#fragment]

where the authority component consists of three more components which are:

authority = [userinfo@] host [:port]

A syntax diagram can be modelled to represent the components of Generic Syntax[18] of URI along with their hierarchy as shown in Figure 2.13.

Consider the following Uniform Resource Locator (URL). Figure 2.14 shows its components.

https://parita@www.semanticweb.org:695/ontology/?tag=survey&order=newest#top

[16]https://www.w3.org/wiki/URI
[17]https://www.w3.org/2001/tag/doc/SchemeProtocols.html
[18]https://www.w3.org/Addressing/URL/uri-spec.html

Term	Definition
Uniform	There are various benefits of achieving uniformity, like, it facilitates different procedures to access resources in the same environment using different types of resource identifiers; new types of resource identifiers can be introduced without modifying the actual usage of the existing identifiers; common syntactic conventions can be interpreted uniformly for a variety of resource identifiers on the semantic web; and the new applications or protocols can utilize the already existing large and widely-used resource identifiers as the identifiers can be reused in different contexts.
Resource	Anything on the web that can be uniquely identified is referred to as a resource. Some day-to-day examples of a resource are a computerized file or folder, weather-broadcasting service, an image, and a set of other resources. Some resources like books in a library, an animal, a human being,and a corporation are a set of resources that cannot be retrieved over the network. A resource is the "conceptual mapping" [11] to a set of entities independent of the corresponding entities at a particular time. Hence, if the conceptual mapping isn't changed, a resource can remain constant irrespective of the changes in its content.
Identifier	An object which is used as a reference to a thing with unique identity on the web is termed as an identifier. In URI, a string of ASCII characters restricted by a specific syntax is object.

Table 2.3: Definitions Used for Characterizing URI, by Berners-Lee et al. [11]

2.4 Linked Data Life Cycle

The Linked Data Life Cycle includes multiple stages as described by Ngomo et al. [36] which are listed in Table 2.4. Each stage of this life cycle is independent yet linked to one another. So, it is not mandatory to go through all the stages sequentially to publish the data on the web. It is necessary to use a procedure that follows the life cycle to ensure the quality of data to be published as linked data. The first stage, generally, is data extraction along with conversion of the extracted data (in forms other than RDF) into RDF. Once the data is in RDF form, any or all of the other stages listed in Table 2.4 can be performed.

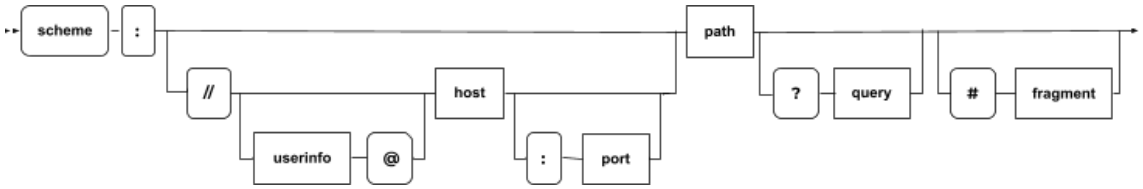

Figure 2.13: Syntax Diagram of a Generic URI, after Berners-Lee et al. [12]

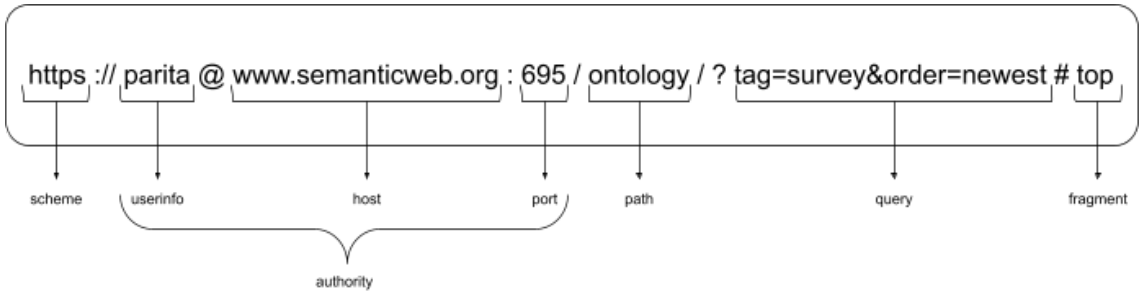

Figure 2.14: Components of an Example of URL, after Berners-Lee et al. [12]

Stage	Description
Extraction	Extraction and conversion of unstructured and semi-structured data into RDF
Storage & Querying	Storing the RDF data in a format that supports data querying efficiently
Authoring	Allowing the users to create new data or to access and modify the existing data
Linking	Creating links among the data belonging to various domains and users based on their entities
Enrichment	Data enrichment for efficient query support using various top-tier structures
Quality Analysis	Managing data using different strategies to ensure good quality of available data on the web
Browsing & Exploration	Making the structured data available on the web for the users to browse and explore effectively

Table 2.4: Stages in Linked Data Life Cycle, after Ngomo et al. [36]

2.5 Ontology in Web Semantics

Ontology is a branch of information science that encloses representation, nomenclature, and definition of various properties, categories, and relationships shared among

the entities[19] that support any number of domains the user is interested in. In other words, Ontology is a form of defining a set of concepts for representing the subject properties and the relationships shared among them. According to Tom Gruber [26], ontology is "an explicit specification of a conceptualization", where conceptualization is "an abstract, simplified view of the world (includes objects, concepts and other entities that are assumed to exist in some area of interest and the relationships that hold among them) that we wish to represent for some purpose". In web semantics, an ontology is used to define the semantics of the resources briefly and methodically. The ontologies have have been used to "specify the physical as well as conceptual characteristics of resources, in the form of metadata schema on the semantic web, for a fixed set of users" [31].

2.6 OWL Ontologies

W3C developed a standard ontology language called OWL[20] to define and describe concepts for ontology using different facilities and operations like intersection, union and negation. It is based on a logical model due to which a reasoner[21] can be used to check the mutual consistency among the statements and definitions in the ontology. The reasoner is also responsible for the identification of a suitable concept for particular definitions [30].

2.6.1 Components of OWL Ontologies

An OWL ontology consists of a set of components that define and describe the concepts collectively. The basic components of any OWL ontology, Individuals, Properties

[19] includes all types of data, concepts and entities
[20] http://www.w3.org/TR/owl- guide/
[21] maintains hierarchy accurately

and Classes, are briefly explained in this section.

2.6.1.1 Individuals

According to Horridge et al. [30], "Individuals represent objects in the domain in which we are interested". Individuals are the objects of classes. Figure 2.15 shows the representation of different individuals in various domains. The triangles are used here to denote the individuals in free space.

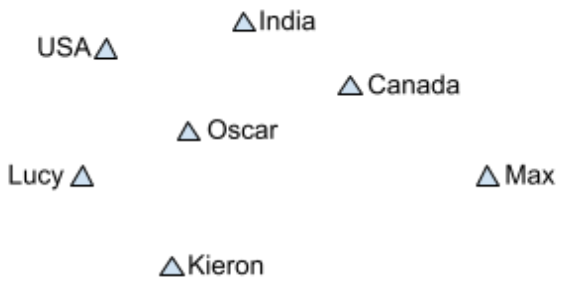

Figure 2.15: Representation of Individuals in OWL

2.6.1.2 Properties

The Binary Relationships[22] between the individuals (things) are called properties. For instance, the individuals "Max" and "Kieron" from Figure 2.15 are linked to each other by the property "hasSibling". Each property can have its inverse. For instance, "hasSibling" can have an inverse property "isSiblingOf". The properties can be symmetric or transitive. It is possible to have functional properties where the properties can be restricted to having only one value.

Figure 2.16 shows representation of properties between the individuals "Max" and "India", and "Max" and "hasSibling". The relation between "Max" and "India" is

[22]links shared by two individuals

"livesIn". The arrow between two individuals in the figure denotes the property (relationship) between them.

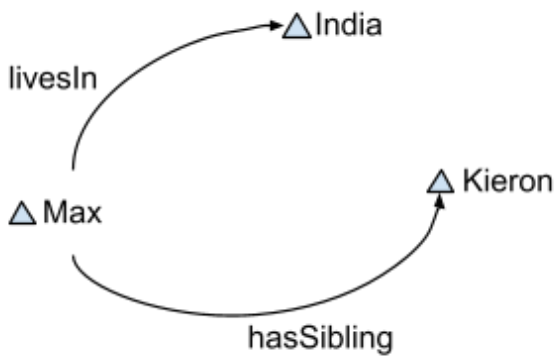

Figure 2.16: Representation of Properties in OWL

2.6.1.3 Classes

In OWL, the sets consisting of individuals are called OWL classes. Mathematical (formal) descriptions are used to describe the OWL classes. These descriptions include accurate statements on the requirements for the class membership. For instance, the class "Country" consists of all the individuals that are countries in the Domain of Discourse[23]. Inheritance in OWL classes (Class Hierarchy) may exist with the classes having subclasses and/or super-classes which is technically termed as taxonomy. "Subclasses specialise ('are subsumed by') their super-classes" [30]. For instance, consider the classes "Lady" and "Human", where "Lady" is a subclass of "Human". This implies the following statements:

- All ladies are humans
- All members of class "Lady" are members of class "Human"
- Being a "Lady" implies that individual is "Human"

[23]can contain one or more classes

- "Lady" is subsumed by "Human"

Figure 2.17 shows the representation of the classes for the individuals and properties that are shown in Figure 2.16.

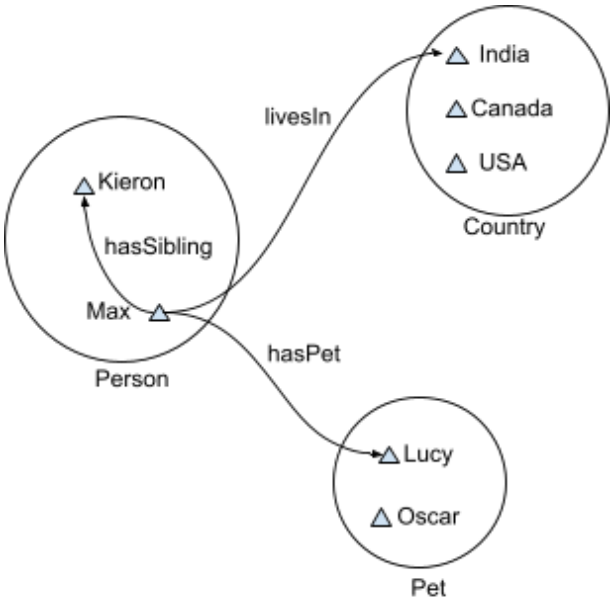

Figure 2.17: Representation of OWL Classes Consisting of Individuals and Properties

2.6.2 Ontology Editors

Ontology editors are used by publishers to create new ontologies when the existing vocabulary is not suitable for their purpose. W3C[24] has recommended a list of ontology editors that can be used to edit, manage, organize and visualize ontology. The list of editors includes Protégé, NeOn Toolkit, SWOOP, Neologism, TopBraid Composer, Vitro, Knoodl, Anzo for Excel, OWLGrEd, Fluent Editor, Semantic Turkey, VocBench. The page does not include valid links to the editors' websites. Protégé is the most popular ontology editor among researchers. There exists negligible information about most of the editors in the list which has driven the decision of the use of protégé

[24]https://www.w3.org/wiki/Ontology_editors

for ontology development and OntoGraf and WebVOWL for visualization which are discussed in detail in chapter 4.

2.7 Conversion of Data from Spreadsheets to RDF

The preferred format of the data published on the semantic web is RDF format. Many data scientists use tables and spreadsheets to store and publish data using Comma Separated Value (CSV) files over RDF format as conversion of data to RDF is comparatively difficult. The data used by Dr. Hepting has been stored in tabular format which can be converted into RDF using different open source tools [23] recommended by W3C[25] or using RDF Mapping Language (RML)[26].

2.7.1 Open-source Tools Recommended by W3C

- OpenRefine:

  OpenRefine[27], an RDF extension of Google Refine, is a powerful java-based set of tools that work with messy data using Google Refine Expression Language (GREL)[28]. It allows the users to extract, transform, export, and convert CSV, Ex cel, spreadsheets and other tabular data into RDF where a Graphical User Interface (GUI) is used to define schema mapping.

- RDF123:

  RDF123[29] is a java application and servlet for the conversion of CSV and spreadsheets to RDF which uses arbitrary graphs to define schema mapping. According

[25]https://www.w3.org/wiki/ConverterToRdf
[26]https://d2s.semanticscience.org/docs/convert-rml/
[27]https://openrefine.org
[28]https://libjohn.github.io/openrefine/grel.html
[29]https://ebiquity.umbc.edu/resource/html/id/237/RDF123-java-application-v1-0

to Han et. al [27], it uses a graphical RDF123 template to allow users to convert the data from spreadsheets to RDF.

- csv2rdf4lod:

  csv2rdf4lod[30] is a tool that is used for the conversion of tabular data into structured and linked RDF using specifications that are based on declarative RDF enhancement parameters. It forms default namespaces for URIs and provides VoID and original metada for the conversion using identifiers for source organization, dataset and version.

- XLWrap:

  XLWrap[31] is a graphical template based application that allows execution of various SPARQL queries [32]. The spreadsheets are wrapped to arbitrary RDF graphs which allows:

  - Streamed processing of tabluar data like Excel, OpenDocument and CSV
  - Loading Local/HTTP
  - Expressions that include calculator operations for Excel and OpenDocument, and Custom functions
  - Use of Application Programming Interface (API) and SPARQL endpoint

- Tarql:

  Tarql[32] operates as a command-line application for the conversion of CSV to RDF using a user-defined mapping that is written in SPARQL standard 1.1.

[30]https://github.com/timrdf/csv2rdf4lod-automation/wiki
[31]https://xlwrap.sourceforge.io
[32]https://tarql.github.io

2.7.2 Conversion Using RML

"An RML mapping defines a mapping from any logical source to RDF. It is a structure that consists of one or more triples maps."[33] The RML rules can be written and tested using tools like the Matey Web UI [34].

2.7.2.1 Tools for RML Conversion

Once the RML rules have been defined, the next step is generate RDF from the RML mappings. The list below shows multiple tools, suggested by Data2Services[35], with various efficiency, stability, and features that can be used for RML conversion.

- RMLMapper:

  RMLMapper[36] is a reference implementation written in Java. It supports custom functions and operations with small files as it loads all data in memory.

- RMLStreamer:

  RMLStreamer[37], written in Scala, is well-suited for large input files and continuous data streams like sensor data. It is still under development resulting in limited support for the use of custom functions.

- SDM-RDFizer:

  SDM-RDFizer[38], written in Python, implements optimized data structures and relational algebra operators that enable an efficient execution of RML triple maps

[33] https://rml.io/specs/rml/#example-XML
[34] https://rml.io/yarrrml/matey/
[35] https://d2s.semanticscience.org/
[36] https://github.com/RMLio/rmlmapper-java
[37] https://github.com/RMLio/RMLStreamer
[38] https://github.com/SDM-TIB/SDM-RDFizer

even in the presence of Big data. A separate tools called Dragoman[39] can be used for the execution of custom functions.

- Morph-KGC:

  Morph-KGC[40] is an RML conversion tool written in Python. It does not support custom functions.

- RocketRML:

  RocketRML[41] is a JavaScript implementation for RML conversion. It provides an easy way to define custom functions.

2.8 Applications of Linked Open Data

A variety of projects and websites, that use linked open data, have been developed. Wikidata and DBpedia are two major projects of linked open data that present structured data on the semantic web. This section consists of a brief description of both applications.

2.8.1 Wikidata

According to a report by Krotzsch and Vrandecic [40], Wikidata was introduced to the world of information engineering in 2012 by Wikimedia. Wikidata can be defined as "the community-created knowledge base of Wikipedia and the central data management platform for Wikipedia and most of its Sister Projects[42]", according to Erxleben et al. [24]. Wikidata was established for the collection and integration of data on the semantic web with Wikipedia. Wikidata is a free open-source tool that handles a large

[39] https://github.com/SDM-TIB/Dragoman
[40] https://morph-kgc.readthedocs.io/en/latest/documentation/
[41] https://github.com/semantifyit/RocketRML
[42] Wikipedia, Wikivoyage, Wiktionary, Wikisource, and others

amount of structured data and maintains pages that store the data. Each entity on Wikidata represents the subject of a triple with a dedicated page. The properties of Wikidata are similar to that of RDF where the individuals and classes are indicated using items. As Wikidata is an open-source tool, the data on a page of the Wikidata is easily available for the users to access as well as edit.

For instance, the University of Regina, a university in the city of Regina (SK, Canada), has an item page allotted for it. Figure 2.18 shows the Wikidata item page of the University of Regina (https://www.wikidata.org/wiki/Q3104287).

Wikidata uses a method of automatically assigning unique identifiers to the entities, which are used as the labels of the item pages instead of the actual names of the entities. These identifiers are a string starting with a capital Q followed by a sequence of digits. The identifiers are assigned irrespective of the language as the Wikidata website supports multiple languages. For the University of Regina, the label used is "Q3104287". The item page on Wikidata consists of multiple sections:

- label: The name of the subject that is treated as an entity. For the instance in Figure 2.18, the label is "University of Regina" along with the unique identifier "Q3104287".

- description: This section consists of general introductory details about the subject. For the University of Regina, the description on wikidata is "university in Saskatchewan, Canada".

- statements: This section consists of a list of subsections called "property". The properties along with their property IDs for the University of Regina are listed in Table 2.5.

- identifiers: This section consists of various IDs belonging to the subject. For the University of Regina, the set of properties used for identifiers and their IDs are

University of Regina (Q3104287)

university in Saskatchewan, Canada edit
Regina College | Regina Campus of the University of Saskatchewan

In more languages
Configure

Language	Label	Description	Also known as
English	University of Regina	university in Saskatchewan, Canada	Regina College Regina Campus of the Universit…
Canadian English	No label defined	No description defined	
French	université de Regina	No description defined	Université de Régina Universite de Regina Université de regina
Italian	No label defined	No description defined	

All entered languages

Statements

instance of
university edit
0 references
add reference

open-access publisher edit
1 reference
stated in Directory of Open Access Journals
add reference
add value

image

edit
University of Regina Library viewed from the Oval.jpg
3,222 × 2,250; 662 KB
0 references
add reference
add value

inception
1911 edit
1 reference
imported from Wikimedia project French Wikipedia

Figure 2.18: Details of University of Regina, Regina, SK on Wikidata

Statement Property	Property ID
instance of	P31
image	P18
inception	P571
country	P17
located in the administrative territorial entity	P131
coordinate location	P625
subsidiary	P355
official website	P856
commons category	P373
topic's main category	P910
category for employees of the organization	P4195
category for alumni of educational institution	P3876
API endpoint	P6269
social media followers	P8687

Table 2.5: Statement Properties and Property IDs for the University of Regina on Wikidata

listed in Table 2.6.

The URI for an entity on Wikidata is "https://www.wikidata.org/wiki/<id>", where <id> stands for the unique identifier. For the University of Regina, the <id> is the label identifier—Q3104287. Figure 2.19 shows the RDF/XML description of the University of Regina on Wikidata.

2.8.2 DBpedia

Jimmy Wales and Larry Sanger, in 2001, created the largest general reference work on the internet consisting of articles[43] in over 280 languages, the Wikipedia[44], which acts as a source of data for many web projects like DBpedia[45], one of the major linked open data projects. The main aim behind the development of DBpedia is the conversion

[43]include data comprising of templates, images, videos, categorizations, and links to other articles (pages) in multiple languages

[44]https://en.wikipedia.org/wiki/Main_Page

[45]https://wiki.dbpedia.org/

```
<?xml version=”1.0” encoding=”utf-8” ?>
    <rdf:RDF xmlns:rdf=”http://www.w3.org/1999/02/22-rdf-syntax-ns#”
        xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
        xmlns:ontolex="http://www.w3.org/ns/lemon/ontolex#"
        xmlns:wdt="http://www.wikidata.org/prop/direct/"
        xmlns:wdtn="http://www.wikidata.org/prop/direct-normalized/">
    <rdf:Description rdf:about="https://www.wikidata.org/wiki/Special:EntityData/Q3104287">
        <rdf:type rdf:resource="http://schema.org/Dataset"/>
        <schema:about rdf:resource="http://www.wikidata.org/entity/Q3104287"/>
        <cc:license rdf:resource="http://creativecommons.org/publicdomain/zero/1.0/"/>
        <rdf:Description rdf:about="https://en.wikipedia.org/wiki/University_of_Regina">
            <rdf:type rdf:resource="http://schema.org/Article"/>
            <schema:about rdf:resource="http://www.wikidata.org/entity/Q3104287"/>
            <schema:inLanguage>en</schema:inLanguage>
            <schema:isPartOf rdf:resource="https://en.wikipedia.org/"/>
            <schema:name xml:lang="en">University of Regina</schema:name>
            <schema:description xml:lang="en">university in Saskatchewan,
            Canada</schema:description>
            <wdt:P18
            rdf:resource="http://commons.wikimedia.org/wiki/Special:FilePath/University%2
            0of%20Regina%20Library%20viewed%20from%20the%20Oval.jpg"/>
        </rdf:Description>
    </rdf:Description>
</rdf:RDF>
```

Figure 2.19: Wikidata Entry (RDF/XML) for University of Regina, Regina, SK

Identifier Property	Property ID
VIAF ID	P214
ISNI	P213
GND ID	P227
Library of Congress authority ID	P244
WorldCat Identities	P7859
ARWU university ID	P5242
Crossref funder ID	P3153
Facebook ID	P2013
Freebase ID	P646
Google Maps Customer ID	P3749
GRID ID	P2427
HAL structure ID	P6773
LittleSis organization ID	P3393
Microsoft Academic ID	P6366
Ringgold ID	P3500
ROR ID	P6782
Times Higher Education World University ID	P5586
Twitter username	P2002
U-Multirank university ID	P5600

Table 2.6: Identifier Properties and Property IDs for University of Regina on Wikidata

of unstructured data on Wikipedia into structured datasets. DBpedia enables querying the Wikipedia data as well as the generation of links among the structured datasets, resulting in the establishment of a massive web of data.

Initially, the template of the page is discovered using a Template Extraction Algorithm[46] which results in the identification of the structure of the template using various Pattern Matching Techniques[47]. Once the appropriate template is selected, the template data is parsed using a suitable algorithm and transformed into RDF triples. MediaWiki[48] links are identified and assigned specific URIs. On the other hand, the common units of data are recognized and assigned appropriate datatypes whereas RDF

[46]like Principle Components Analysis, Linear Discriminant Analysis, Logically Linear Embedding

[47]https://www.dummies.com/programming/big-data/data-science/how-pattern-matching-works-in-data-science

[48]https://www.mediawiki.org/wiki/MediaWiki

lists are created for the object lists. According to Auer et al. [4], DBpedia in 2007 was a platform that delivered information on "more than 1.95 million 'things' that includes at least 80,000 persons, 70,000 places, 35,000 music albums, 12,000 films".

The DBpedia semantic web services are handled using the GNU Free Documentation License terms and policies. The data on DBpedia is easily accessible in different ways like accessing the datasets as linked data; using SPARQL queries for data extraction and accessibility; and in the form of a downloadable RDF snippets [18]. Figure 2.20 shows DBpedia entry for the "University of Regina, Regina, SK". Like Wikidata, DBpedia also has a label, a description section and a set of important property-value pairs that give information related to the entity that has been described.

Figure 2.21 shows the data architecture of DBpedia. An open-link software, Virtuoso Universal Server[49], is a data virtualization tool used to host and publish RDF datasets on DBpedia, which can be easily accessed using SPARQL endpoint. The HTML or RDF representation of the DBpedia resources is enabled by the data architecture using HTTP that supports the standard HTTP GET method calls from the web clients.

[49]https://dbpedia.org/page/Virtuoso_Universal_Server

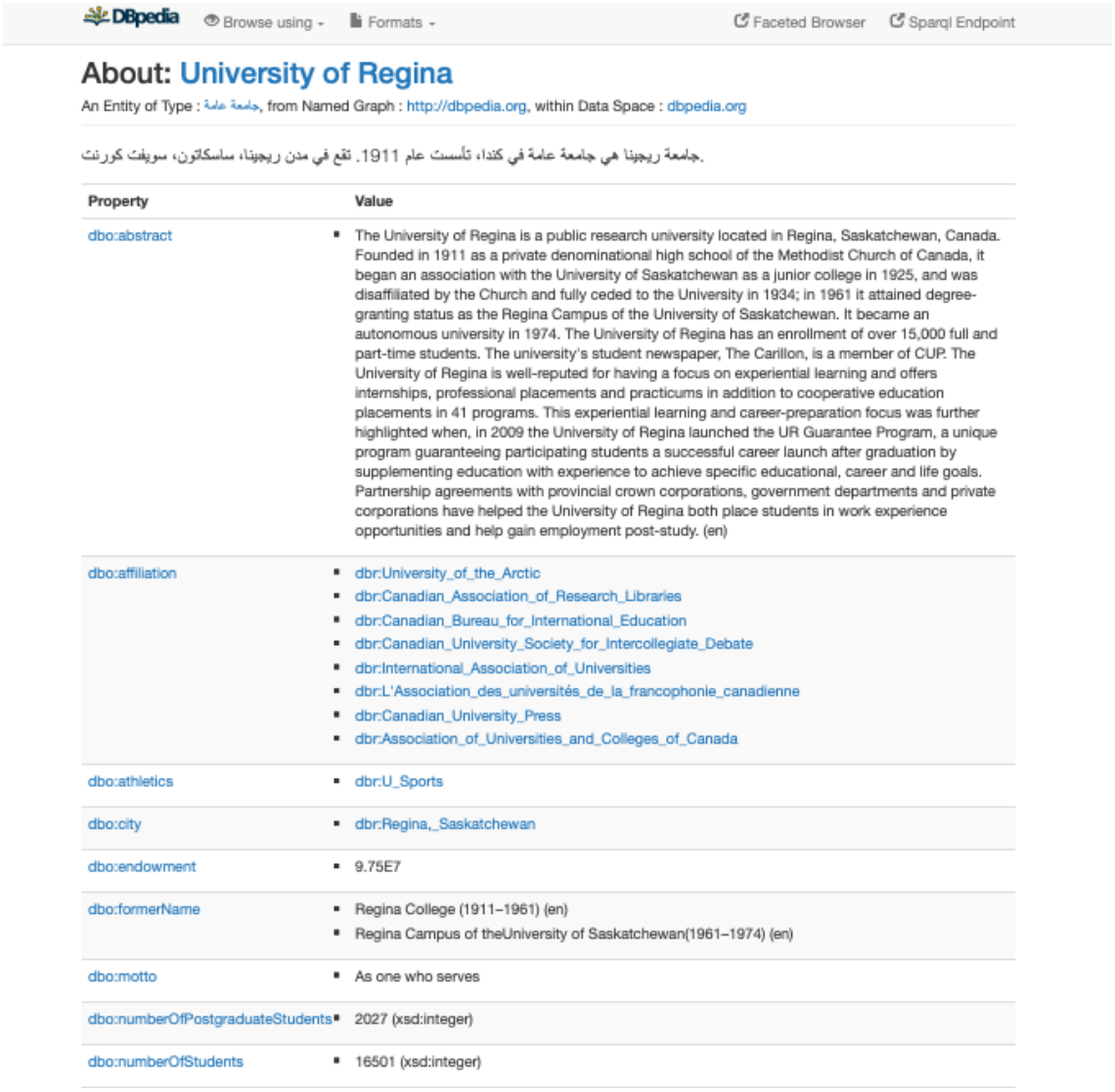

Figure 2.20: Description of University of Regina, Regina, SK on DBpedia

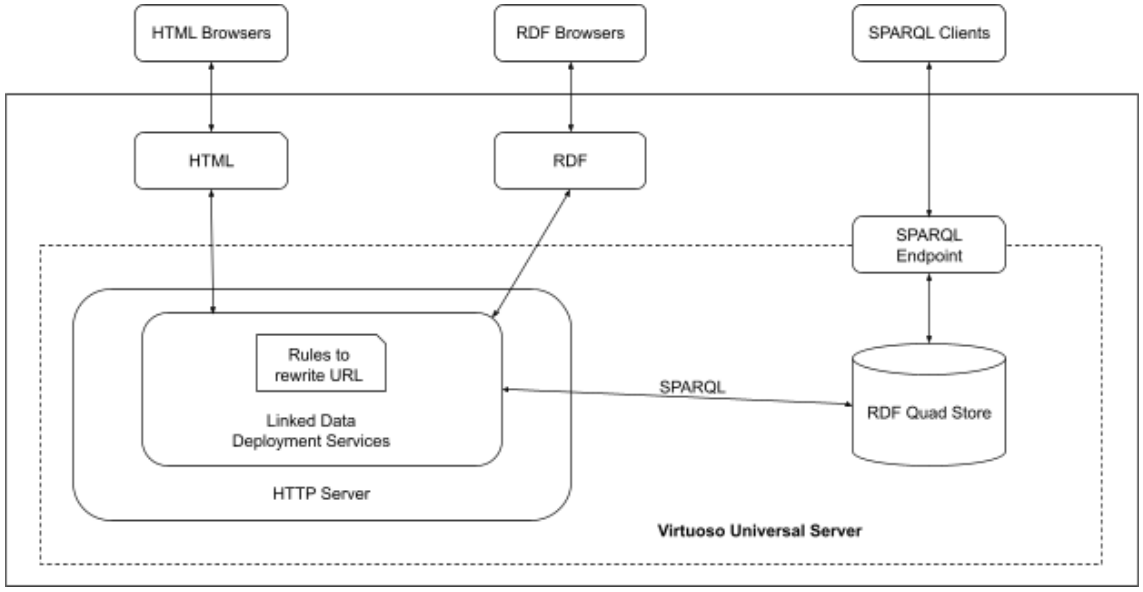

Figure 2.21: DBpedia Data Architecture

Chapter 3

Methodology

This chapter focuses on the usability of protégé along with the methodology involved in describing and publishing linked open data. Chapter 4 presents the proof of concept for this methodology.

3.1 Describing New OWL Vocabularies

In data science, for a fresh and developing branch like ontology engineering, it is difficult to find enough knowledge that can be reused. The field is still growing with research and development in progress as well as a lot still required to be explored in the future. As a result of this, linked open vocabularies are limited currently, and hence, the data publishers will have to create a vocabulary in case they do not find the required vocabulary from the already existing vocabularies.

There are editors available today that help the publishers to create their own linked open vocabularies. Alatrish at the University of Belgrad [2], in 2013, compared the most used ontology editors, namely, Apollo, OntoStudio, Protégé, Swoop, and TopBraid Composer. The editors were compared based on the following features:

- General description: developers' details, and accessibility on the internet (open source or software licensed)

- Software architecture and tools evaluation: semantic web architecture, support for plug-ins, backup management, and ontology storage
- Interoperability: interoperability with other ontology tools, support and translation for other languages
- Knowledge representation: Knowledge Representation (KR) paradigm of knowledge model, axiom language, and methodological support
- Inference services: built-in inference engine, other attached inference engines, and constraint/consistency checking
- Usability: graphical taxonomy, graphical views, zooms, collaborative working, and ontology libraries

After studying the details and comparison among the most used editors based on the above features by Alatrish [2], protégé has been used for the development of the instrument to support this research and thesis work.

3.1.1 Protégé

Stanford University developed a free, open-source ontology development platform, Protégé[1], that offers a set of substantial tools[2] to build ontologies for knowledge-based applications and to implement various "knowledge-modeling structures and actions that support the creation, visualization and manipulation of ontologies in various representation formats" [2]. Protégé offers customization options to build knowledge models and to add data.

[1]https://protege.stanford.edu
[2]along with various plug-ins and Java-based API

3.1.2 Usability of Protégé

The International Organization for Standardization (ISO) standard 9241[3] defines usability as "the extent to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use". In usability engineering, Jacob Neilson [37] suggested five qualities of a usable product: "learnability, efficiency, memorability, errors (low rate, easy to recover), and satisfaction". These were the original five dimensions of usability which were then re-termed as all five words starting with "E" by Quesenbery [39], and they became popular as the five Es of usability [39]. The new terms are "Effective", "Efficient", "Engaging", "Error-tolerant", and "Easy to learn"

The five dimensions of usability can be considered while designing as well as for evaluation. While designing, the engineers consider the five dimensions as goals to be achieved whereas while testing and evaluation, they are considered as the requirements necessary for the designed model. The five dimensions of usability were used to assess the usability of protégé and the report of the evaluation is as follows:

- Effective: The task is completed accurately

  Protégé is an effective design as it allows the users to build ontologies and use other tools precisely. The users can complete the desired tasks accurately using the set of facilities available.

- Efficient: The task is completed quickly

  It is easy and straightforward to describe the vocabularies and build ontologies. All the desired tasks in protégé can be completed quickly (without wasting much time) as it is easy to look for the facilities using the menus and tools available. Hence, protégé is an efficient ontology builder.

[3]https://www.iso.org/standard/63500.html

- Engaging: The look and feel of the design is interactive and engaging

  The tools and menus in protégé are interactive which leads to a better experience for the user. Also, the layout and the components are attractive and decent for the user to use this ontology builder whenever necessary. Overall, protégé gives an engaging and satisfactory user experience.

- Error-tolerant: Error prevention and data recovery in case any error is committed

  Protégé provides options for modifications, additions, and deletions of classes, objects, and properties, which allow the users to correct the errors that were committed. Also, the deleted classes can be recovered using "Undo" from the edit menu resulting in proving protégé as an error-tolerant design.

- Easy to learn: The interface should be easy to learn

  Protégé is easy to learn as it has a familiar look and commonly used names for the tools and menus making it easy for the users to adapt and know them nicely. Also, the "Help" document along with a list of some frequently asked questions is available which helps the users in the correct direction whenever they are lost.

3.2 Procedure to Describe OWL Vocabularies Using Protégé

This section focuses on a step-wise methodology that has been followed to create new OWL vocabularies using protégé. Protégé is available for use as a web application or it can also be downloaded[4] to use as a desktop application.

To commence with the process of building an ontology, a new project needs to be

[4]https://protege.stanford.edu/products.php#desktop-protege

created with a unique Internationalized Resource Identifier (IRI)[5] and the ontology version. Figure 3.1 shows the protégé home-screen with a new project created and an IRI with the name and version of ontology.

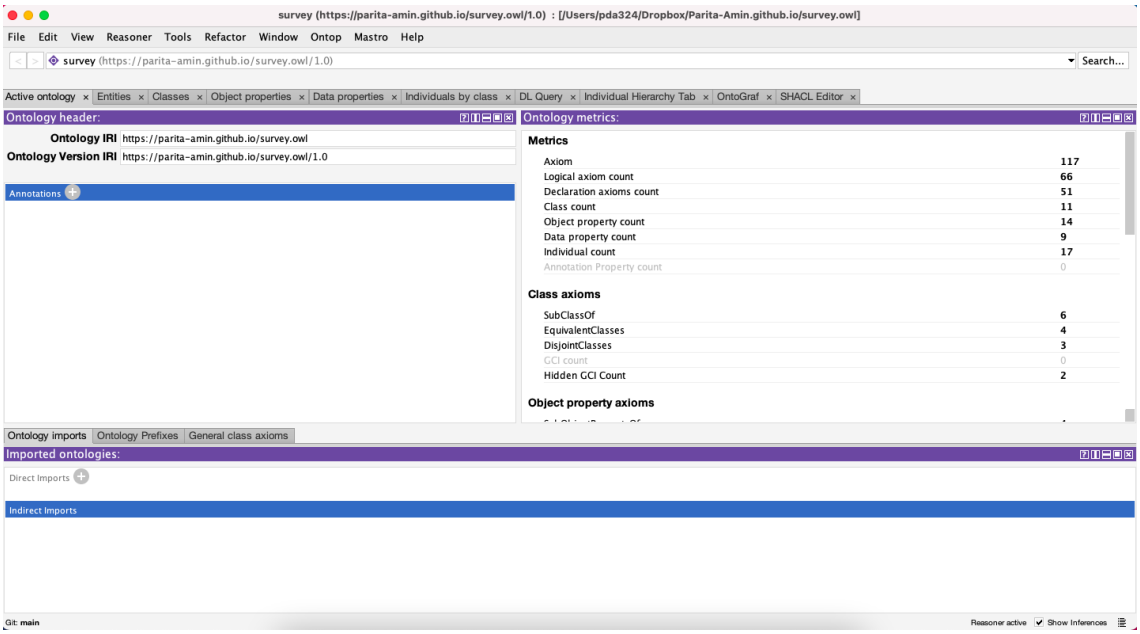

Figure 3.1: Protégé Home-screen for a New Project with an Ontology Name and an IRI

3.2.1 Classes and Class Hierarchy

The next step is to create classes and build the hierarchy for the class nodes. Each entity in protégé is directly or indirectly a child node of the "Thing" class. Also, each object has a name-value pair, which generally are the nodes of the "Domain_Entity" class, an inherited class of Thing. Initially, four classes, "Thing", "Domain_Entity", "Independent_Entity", and "Value" are created using class hierarchy, as shown in Figure 3.2. To create a class hierarchy for the classes that represent entities in an ontology, there are three steps to be followed:

[5]https://www.w3.org/2011/rdf-wg/wiki/IRIs/RDFConceptsProposal

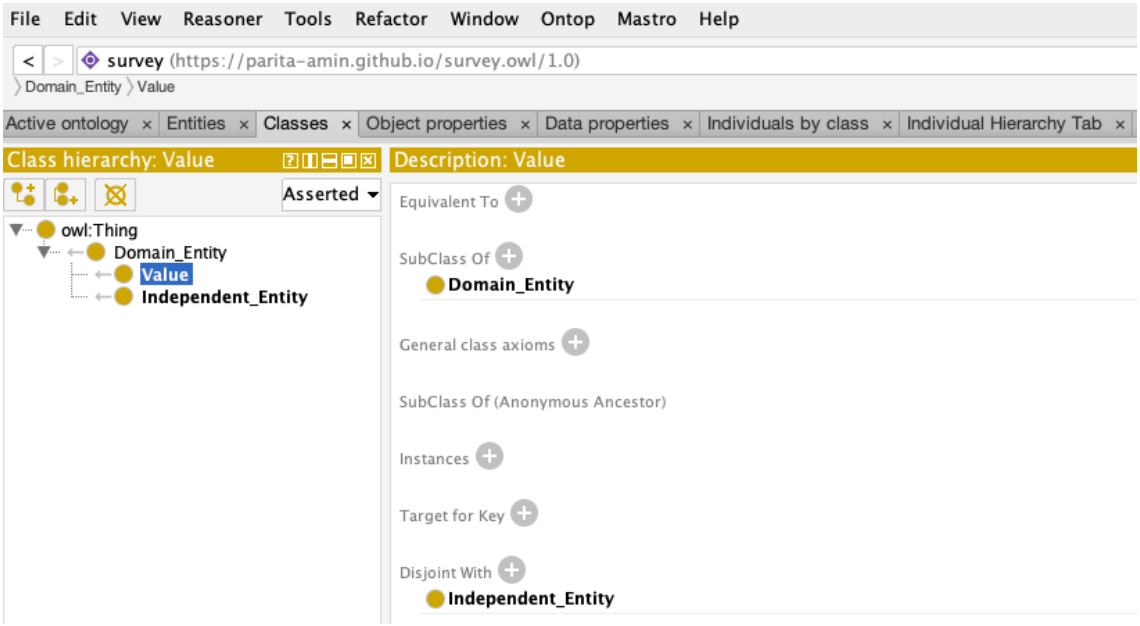

Figure 3.2: Class Hierarchy of Thing, Domain_Entity, Independent_Entity, and Value

1. Select "owl:Thing" in the Class Hierarchy View of the Classes tab from the Windows menu followed by selecting "Create class hierarchy..." from the Tools menu, as shown in Figure 3.3 to open a "Enter hierarchy" pop-up window which allows the users to add classes to the ontology.

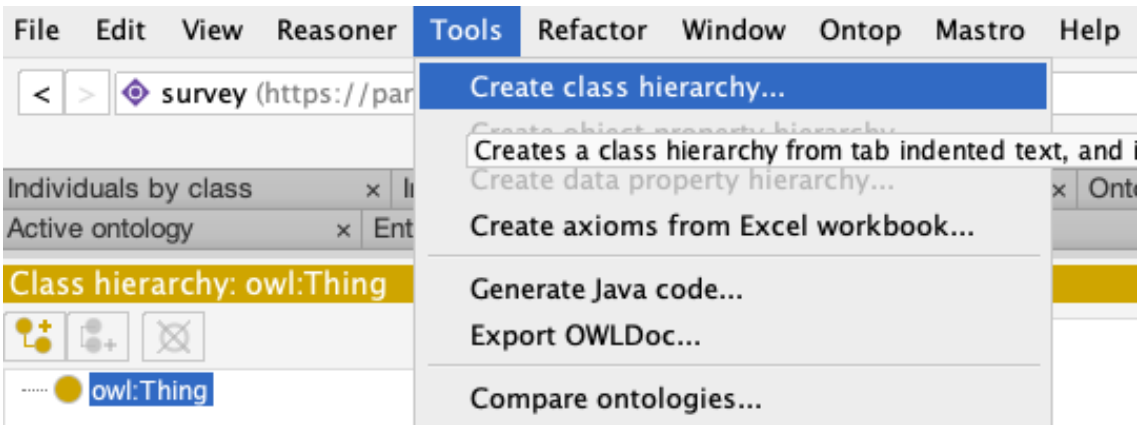

Figure 3.3: Tools Menu to Create Class Hierarchy

2. Enter the class names in the pop-up window starting from the left. For the child nodes, leave some spaces before writing the class name. Each horizontal line

consists of one class name, as shown in Figure 3.4.

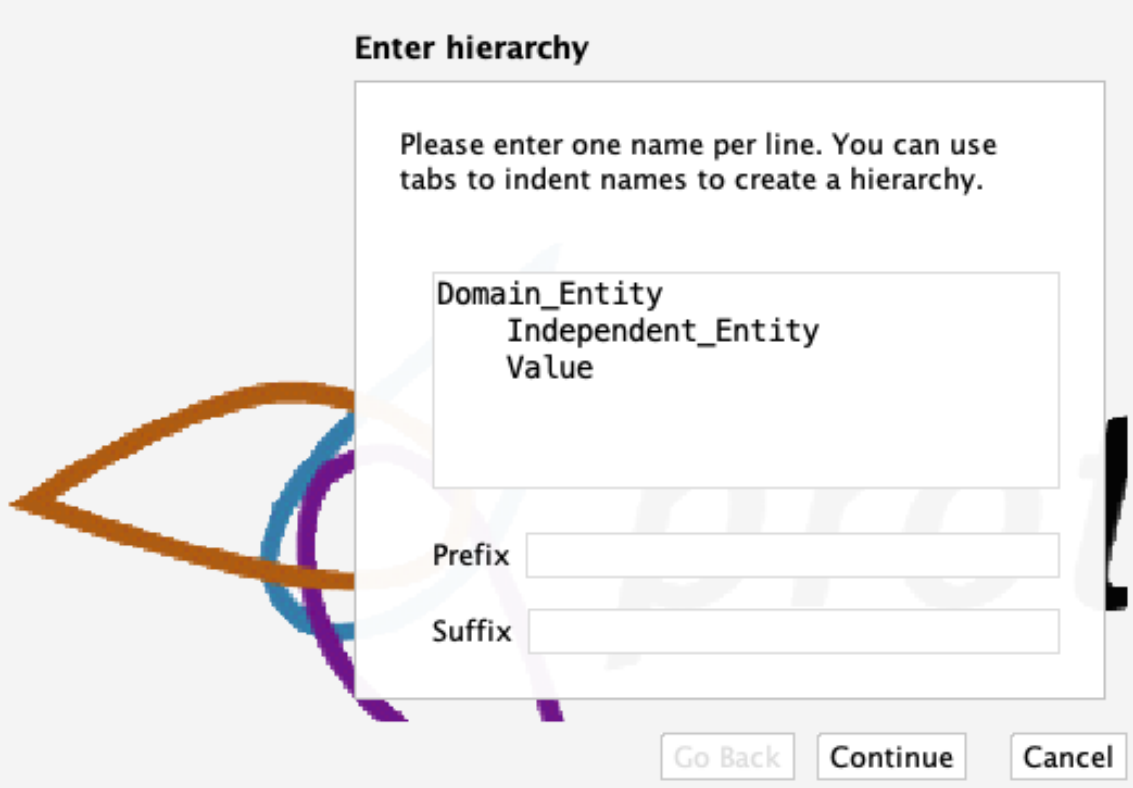

Figure 3.4: "Enter hierarchy" Pop-up Window

3. On clicking "Continue" on the pop-up window in Figure 3.4, it opens up a new pop-up window. Here, it is important to state whether one wants the sibling classes to be disjoint or not. The checkbox needs to be checked accordingly. Usually, it is recommended to create disjoint sibling classes which can be changed later as necessary, as shown in Figure 3.5.

Once the classes are added, the next step is to describe the classes. Descriptions of classes like "Equivalent to", "SubClass Of", "Instances", "Disjoint With" and so on can be added using the Description view of the Class tab (shown in Figure 3.2). Each class can be described individually by selecting a particular class and adding details to the description view.

Figure 3.5: Pop-up Window to Create Disjoint Sibling Classes

3.2.2 Object Properties and Object Property Hierarchy

The next step in building the ontology, after describing the classes, is describing the "Object Properties". Object properties are used to link the entities (classes in protégé). The "owl:topObjectProperty" is the root node of the object properties' hierarchy. The hierarchy for object properties can be built by selecting a property in the Object Property Hierarchy view of the Object Property Views tab and then going to the tools menu as shown in Figure 3.6. The remaining steps are similar to the ones that are followed for building class hierarchy (Figure 3.4 and 3.5).

Also, the description view of object properties hierarchy allows the user to describe the object properties. The details that can be added or modified using the description view are "Equivalent To", "SubProperty Of", "Inverse Of", "Domains (intersection)", "Ranges (intersection)", "Disjoint With", and "SuperProperty Of (Chain)".

The names of the object properties usually start with an "is" or a "has". For instance, consider two classes: "Lady" (class 1) and "Parita" (class 2). To describe the link

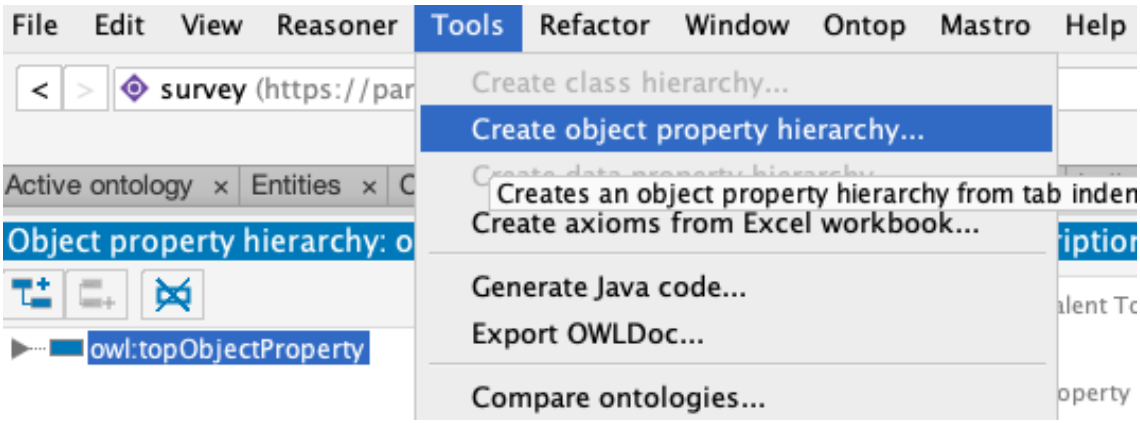

Figure 3.6: Tools Menu to Create Object Property Hierarchy

between class 1 (Lady) and class 2 (Parita) such that class 1 has value class 2 and class 2 is value of class 1, the properties "hasName" and "isNameOf" would be used respectively, as shown in Figure 3.7.

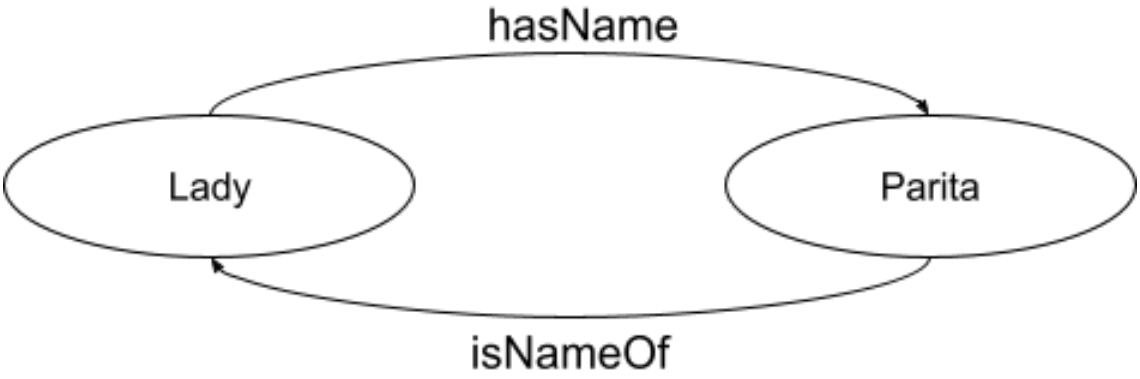

Figure 3.7: Object Properties for Classes "Lady" and "Parita"

Each object property might have a corresponding inverse property. For the above instance, it can be said that:

Lady hasName Parita **is equivalent to** Parita isNameOf Lady

Here, "hasName" is inverse of "isNameOf" and "isNameOf" is inverse of "hasName". Apart from inverse, domain and range[6] are two other important parts of description

[6]http://protegeproject.github.io/protege/views/object-property-description/

of object properties. According to Horridge et al. [30], in OWL, "domain and range are not constraints to be checked. They are axioms which are used by the reasoner to make inferences". As shown in Figure 3.8, "Lady" acts as the "Domain" and "Parita" acts as the "Range" for "hasName" whereas for "isNameOf", "Parita" is the "Domain" and "Lady" is the "Range".

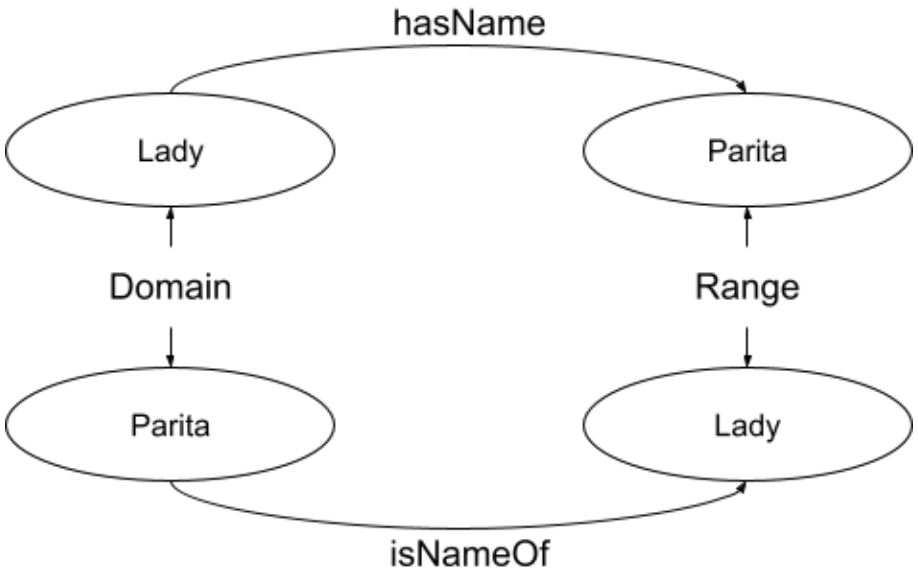

Figure 3.8: Domain and Range Classes for "hasName" and "isNameOf"

3.2.3 Data Properties and Data Property Hierarchy

Furthermore, the "Data Properties" can also be described for ontology. "An ontology data property provides a relation to attach an entity instance to some literal datatype value (an RDF number, string or data for example) that is a measure or estimate of what that data property is about."[7]

In protégé, the "owl:topDataProperty" is the root node of the data properties' hierarchy. The data property hierarchy can be built by selecting a property in the Data

[7]https://ddooley.github.io/docs/data-properties/

Property Hierarchy view of the Data Property Views tab and then going to the tools menu as shown in Figure 3.9. The remaining steps are similar to the ones that are followed for building class and object property hierarchy (Figure 3.4 and 3.5).

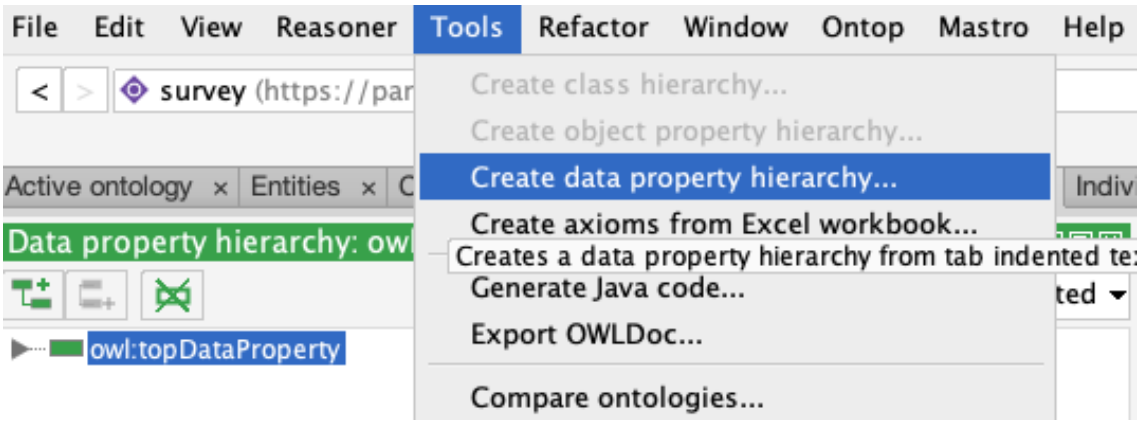

Figure 3.9: Tools Menu to Create Data Property Hierarchy

Data properties can be described using the description view of data properties of hierarchy which included details like "Equivalent To", "SubProperty Of", "Domains (intersection)", "Ranges", and "Disjoint With".

3.2.4 OWL Ontology Inconsistencies and Reasoner

After describing the classes, object properties, data properties, and individuals, the next step is to determine and fix the errors and inconsistencies in the ontology using a Reasoner[8]. The current version of the protégé Integrated Development Environment (IDE) offers support for some built-in semantic reasoners which are listed below:

- ELK 0.4.3[9] : A Java-based fast reasoner for lightweight OWL2 EL which is available under Apache License 2.0

[8]https://en.wikipedia.org/wiki/Semantic_reasoner
[9]https://www.w3.org/2001/sw/wiki/ELK

- FaCT++ 1.6.5[10] : A Tableaux-based reasoner for expressive OWL Description Logics (DL) which is implemented using C++

- HermiT 1.4.3.456[11] : A Java-based reasoner, written using OWL, that can be used to determine consistency of the ontology as well as for the identification of relationships among the classes of the ontology

- Mastro DL-Lite[12] : An Ontology-Based Data Access (OBDA) management system in which the DL-Lite family of languages for lightweight DL are used for ontology specifications

- Ontop 4.1.0[13] : A reasoner which uses SPARQL for querying datasets as Virtual RDF Graphs

- Pellet and Pellet (incremental)[14] : A Java-based OWL2 reasoner for the optimization of nominals, conjunctive query answering, and incremental reasoning

3.2.5 Data Validation

Dr. Hepting collects the feedback responses and manually stores them in spreadsheets. It is important to validate the stored data so that the data only consist of acceptable values. According to Ekaputra and Xiashuo [22], "One of the challenges of adopting Semantic Web Technologies (SWT) for enterprise data integration is to provide the means to define and validate structural constraints over RDF graphs." To address this challenge, they developed SHACL4P, a protégé plugin. SHACL4P is used for defining and validating SHApes Constraint Language (SHACL), which is a W3C

[10]https://www.w3.org/2001/sw/wiki/Fact
[11]https://www.w3.org/2001/sw/wiki/Hermit
[12]https://protegewiki.stanford.edu/wiki/Mastro_DL-Lite_Reasoner
[13]https://www.w3.org/2001/sw/wiki/Ontop
[14]https://www.w3.org/2001/sw/wiki/Pellet

standard for the validation of contents of an RDF-style graph database[15]. SHACL uses "shapes" to describe constraints.

3.3 Conversion of Tabular Data into Linked Open Data

After describing the best possible version of the ontology, the next step is to convert the data stored in spreadsheets into linked open data using either OpenRefine or RML-Streamer. As RMLStreamer is still under development and provides limited support for custom functions, this research uses OpenRefine.

3.3.1 Conversion Using OpenRefine

OpenRefine is a free, open-source powerful tool for working with messy data. OpenRefine can be downloaded and installed from the OpenRefine website[16]; run as a web server; and accessed using local web server in the browser. The first step is to load OpenRefine and access it in the browser. Figure 3.10 shows the homepage of OpenRefine.

OpenRefine offers options to create a new project as well as to open an existing project. The users can open an existing project using "Open Project" and continue working on the data. Creating a new project includes following steps:

1. Click on "Create Project", select the type on sources of dataset and then click on "Choose Files" button to select data sources as shown in Figure 3.11. OpenRefine supports imports from Tab Separated Value (TSV), CSV, JavaScript Object Notation (JSON), XML, RDF/XML, Excel, and Google Data documents whereas the other file types can be supported using OpenRefine extensions.

[15]https://www.w3.org/TR/shacl/
[16]https://openrefine.org/

Figure 3.10: OpenRefine Homepage

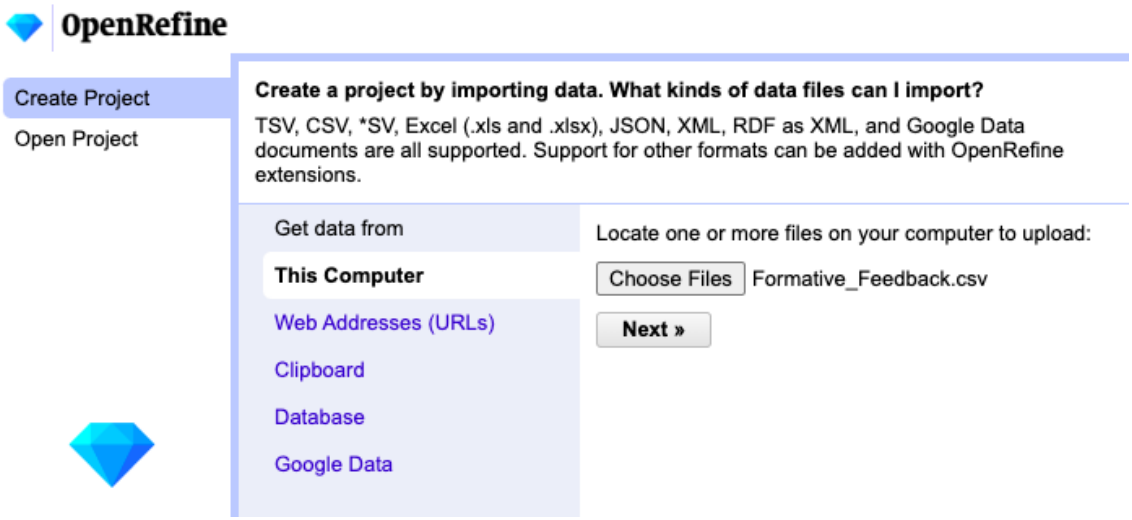

Figure 3.11: Create a New OpenRefine Project

2. The next step is to decide the orientation of the data to be displayed using various options as shown in Figure 3.12. The "Update Preview" button allows viewing the results of these modifications.

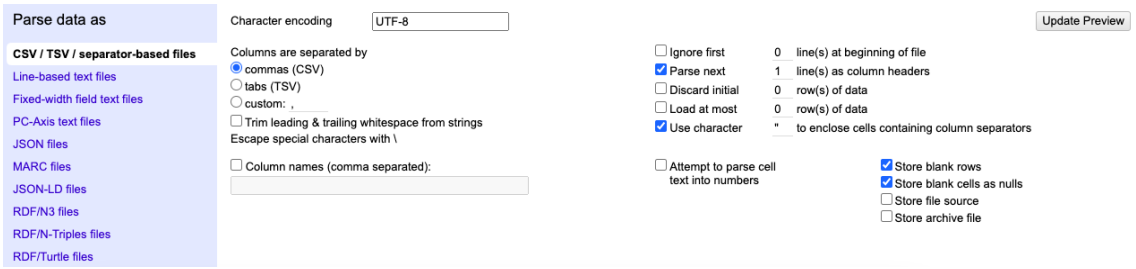

Figure 3.12: Options to Set Orientation of Data in New OpenRefine Project

3. The final step is to mention the name of the project and click on the "Create Project »" button as shown in Figure 3.13.

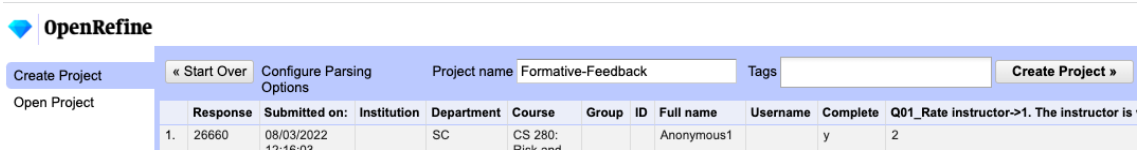

Figure 3.13: Naming the New OpenRefine Project

OpenRefine provides support for following tasks:

1. Exploring Data:

   Large data sets can be explored with ease using OpenRefine. The "Facet" option allows users to explore values of particular type by offering options like "Text facet", "Numeric facet", "Timeline facet", "Scatterplot facet", and other custom facet. Figure 3.14 shows the facet options offered by OpenRefine for a field.

   The left side of the screen in Figure 3.15 shows the result of using "Text facet" for "Response" column. It also offers an option to "edit" and "include" the results of the facet. Figure 3.16 shows the result of using "include" for the record corresponding to the response "26480".

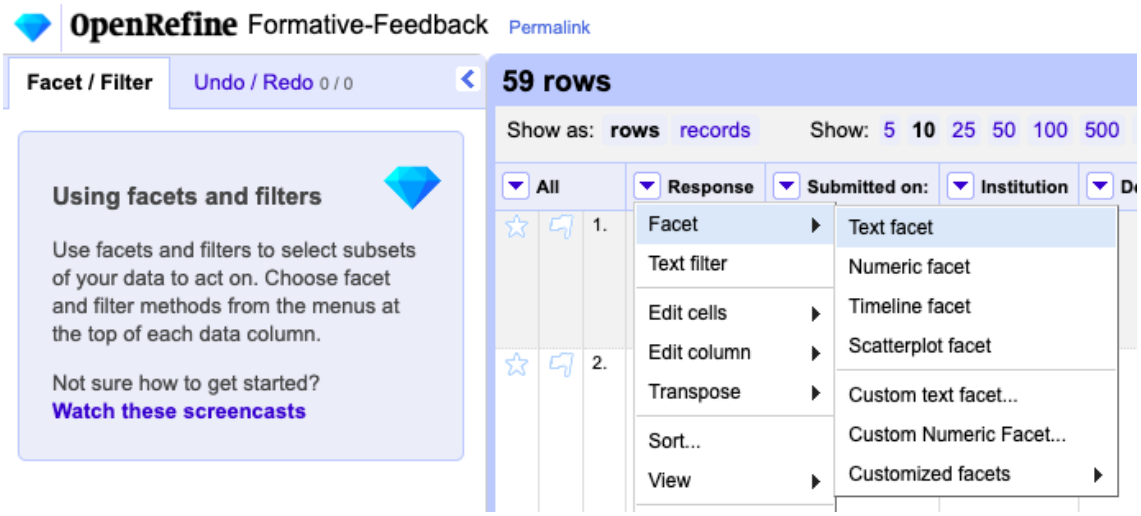

Figure 3.14: Exploring Data in OpenRefine - 1 of 3

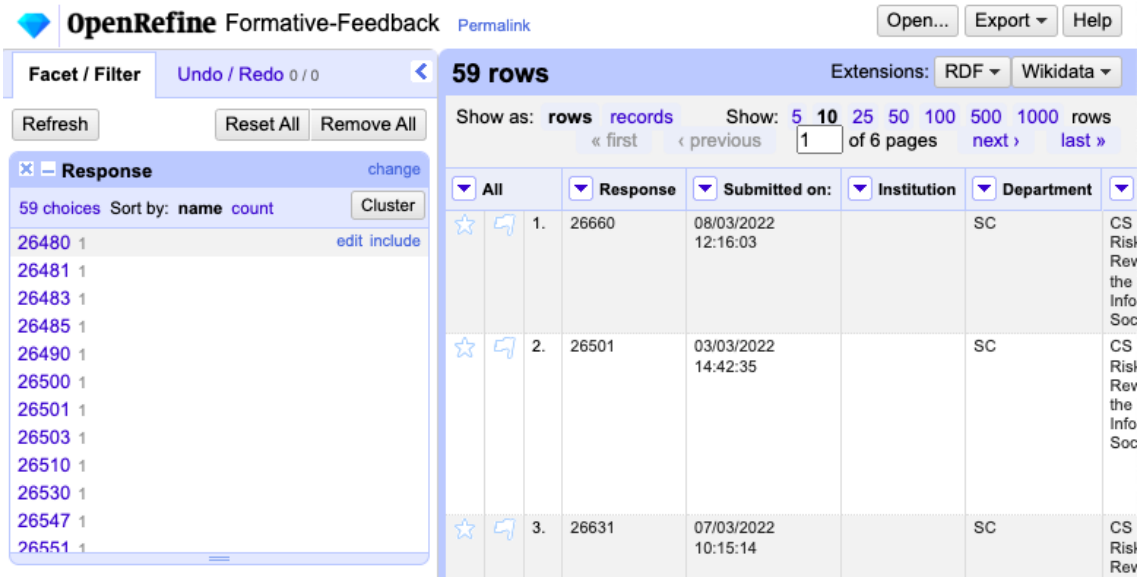

Figure 3.15: Exploring Data in OpenRefine - 2 of 3

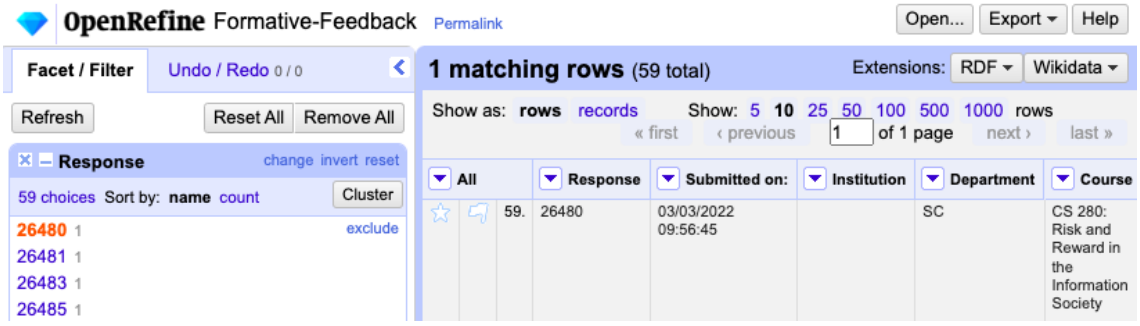

Figure 3.16: Exploring Data in OpenRefine - 3 of 3

2. Cleaning and Transforming Data:

- Cleaning Data:

  While dealing with messy data, OpenRefine can be used to clean the data using the "Text filter" option. Figure 3.17 shows the "Text filter" option offered by OpenRefine for a field.

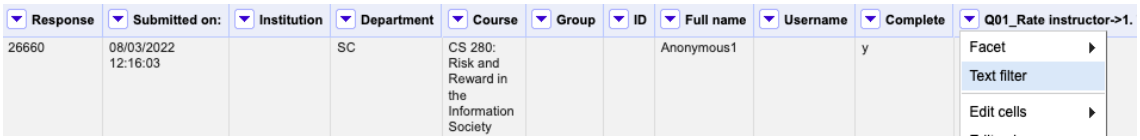

Figure 3.17: Cleaning Data in OpenRefine - 1 of 2

Figure 3.18 shows the result of applying the "Text filter" option for the "Q01_Rate instructor->1." field and using "-999" as argument.

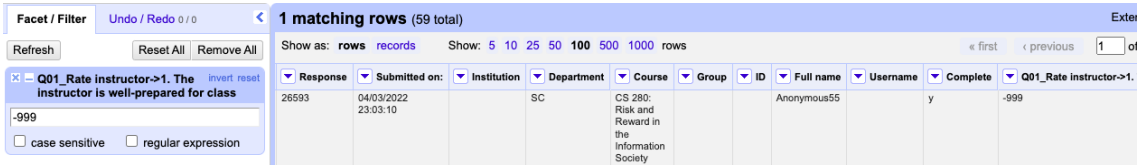

Figure 3.18: Cleaning Data in OpenRefine - 2 of 2

- Transforming Data:

  OpenRefine allows transforming data from one format to another using "Edit cells" option under which "Transform..." and "Common transform" offers

different options for transforming data. Figure 3.19 shows all the options offered by OpenRefine for transforming data.

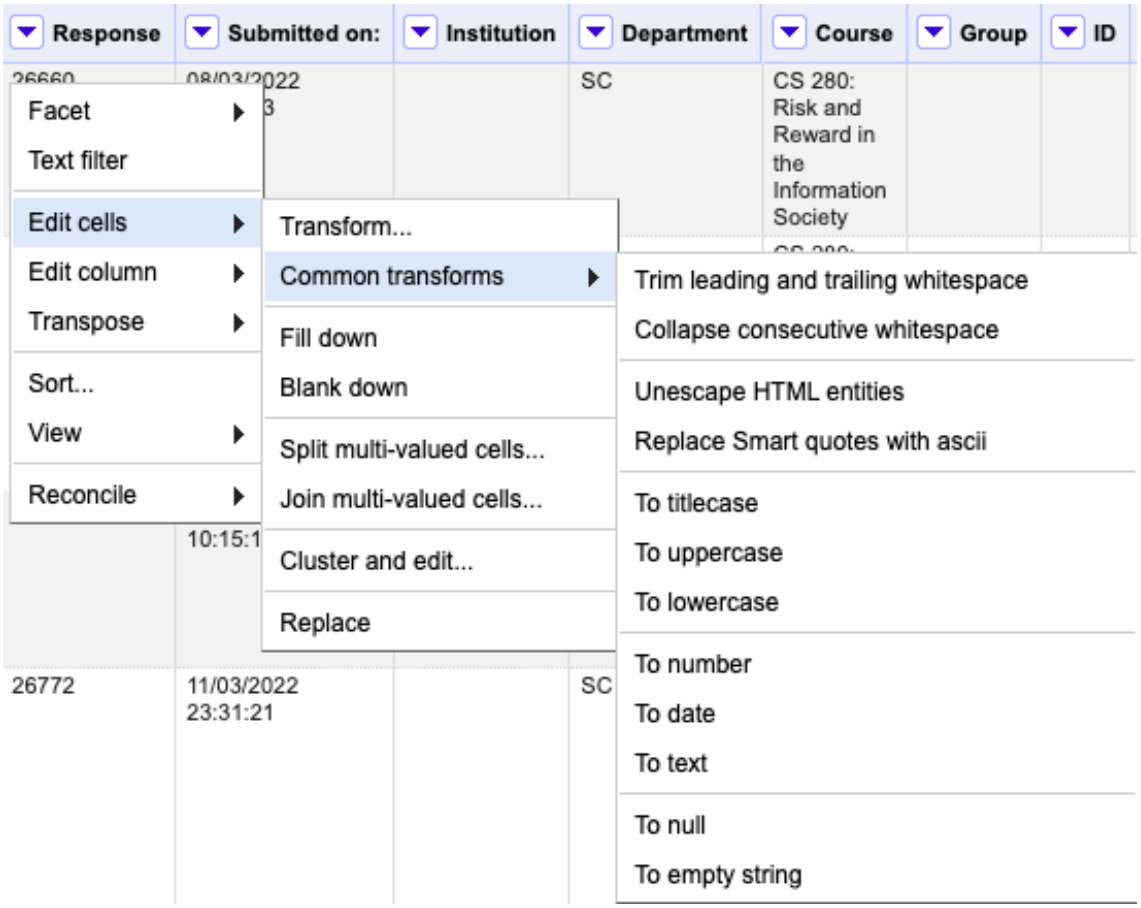

Figure 3.19: Transforming Data in OpenRefine - 1 of 3

Figure 3.20 shows the result of selecting "Transform..." option for "Response" field and modifying the Expression to "Response:" + value. It also offers preview of the result of the GREL expression.

As a result of using the "Custom text transform on column Response" in OpenRefine, the text of all the records for "Response" field would be transformed from one format to another. For instance, the value of "Response" for the first record is transformed from "26660" to "Response:26660" as shown in Figure 3.21.

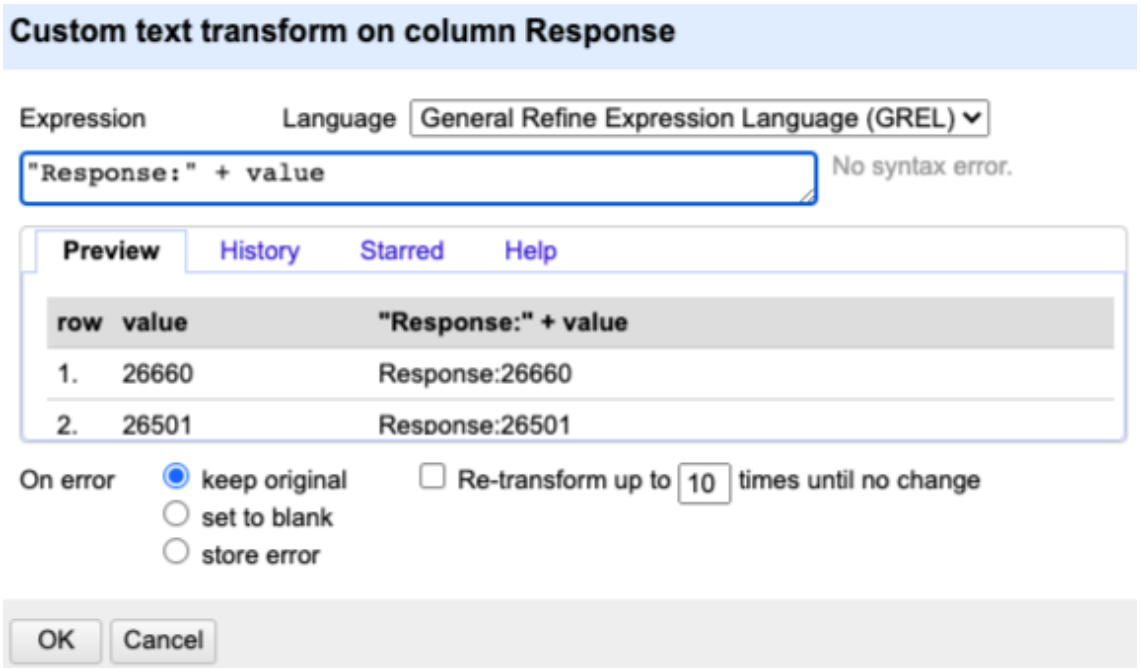

Figure 3.20: Transforming Data in OpenRefine - 2 of 3

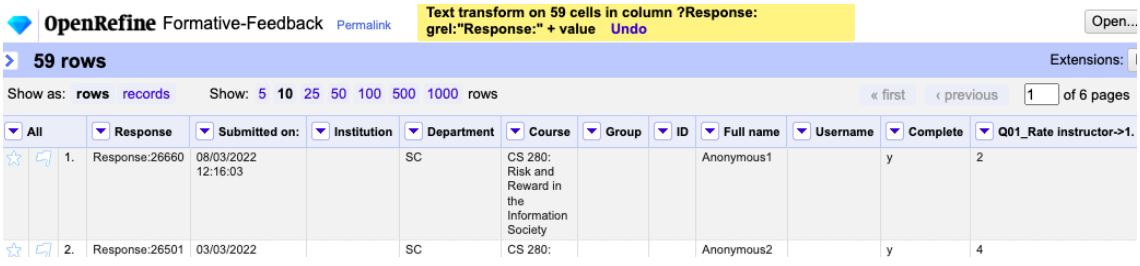

Figure 3.21: Transforming Data in OpenRefine - 3 of 3

3. Reconciling and Matching Data:

   OpenRefine can be used to link and extend dataset with various web services. It also supports services (using extensions and plugins) that can be used to upload the cleaned data to central databases like Wikidata.

3.4 Publishing Linked Open Data

As stated by Petrou et al. [38], according to the principles of linked data, dereferencable URIs can be used to access linked data which the way to access Linked Data is via dereferencable URIs, which, when accessed, provide meaningful descriptions of the concept they represent in a variety of formats, and the data should be accessible to the public. Therefore, access to the data can be provided as RDF dumps or via SPARQL endpoints. To make the data accessible, there are multiple ways

Finally, machine access is provided as it is one of the best practices of publishing Linked Data. The datasets converted can be accessed at http://linked-statistics.gr in three ways: (a) download the data as RDF dumps for local processing, (b) query and browse the data using the SPARQL endpoint service and SPARQL query form, and (c) link to the data by referencing to their unique identifier (URI).

Chapter 4

Proof of Concept

Chapter 3 focuses on the methodology which has been followed to describe and publish the linked open vocabulary for survey in this chapter.

4.1 OWL Ontology for Survey Using Protégé

Figure 4.1 shows the student feedback form for which the new vocabularies are described.

4.1.1 Classes for Survey Ontology

All the classes can be created and described using the steps described in section 3.2.1. The resultant class hierarchy for the survey ontology would be as shown in Figure 4.2. A survey can have "Survey Response", "Respondent", "Response Date", "Type of Survey", "Survey Questions", and "Survey Answers", which are considered for this ontology as well.

Here, the type of survey used is feedback survey ("FeedbackSurvey" class). The "SurveyCollection" class corresponds to a collection of survey responses ("SurveyResponse" class) that were collected for a particular course ("CourseID" class) in a particular semester ("SemesterID" class) with a specific collection ID ("CollectionID" class).

Student Feedback Form

(adapted from http://cft.vanderbilt.edu/teaching-guides/reflecting/student-feedback/#inclass)

Course: _____ **Instructor**: Daryl Hepting **Date**: _____

1 = Never; 5 = Always

1	The instructor is well-prepared for class.	1	2	3	4	5
2	The instructor clearly communicates his expectations for student preparation and participation.	1	2	3	4	5
3	The instructor uses class time effectively.	1	2	3	4	5
4	The instructor has clear expectations for assigned work.	1	2	3	4	5
5	The instructor encourages student participation.	1	2	3	4	5
6	The instructor clearly answers questions.	1	2	3	4	5
7	The instructor treats students with respect.	1	2	3	4	5
8	The instructor effectively directs and stimulates discussion.	1	2	3	4	5
9	The instructor effectively encourages students to ask questions and give answers.	1	2	3	4	5

What do you like best about this course?

What would you like to change about this course?

What do you think the instructor's greatest strengths are?

Figure 4.1: The Student Feedback Form, from Hepting [29]

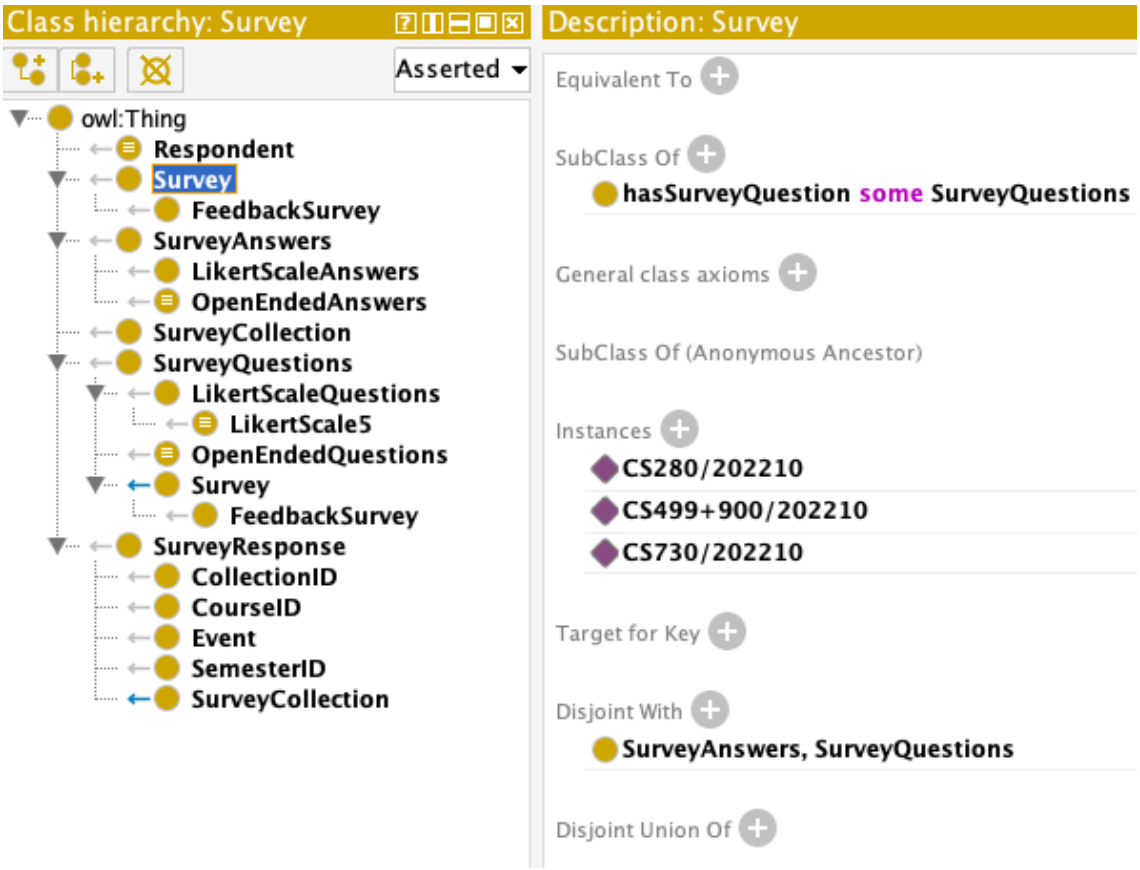

Figure 4.2: Class Hierarchy in Survey Ontology

Furthermore, the survey questions and survey answers have classes corresponding to various forms of question-answer. Generally, feedback surveys consist of two types of questions, namely, "Likert Questions" and "Open-ended Questions". The type of questions might differ depending on the designer of the survey, some of which, as stated on the *SurveyMonkey*[1] website, are listed below.

- Multiple Choice Questions
- Rating Scale Questions
- Likert Scale Questions
- Matrix Questions
- Dropdown Questions
- Open-ended Questions
- Demographic Questions
- Ranking Questions
- Image Choice Questions
- Click Map Questions
- File Upload Questions
- Slider Questions
- Benchmarkable Questions

The feedback survey in this research has Likert and Open-ended questions. The classes described for the questions and answers are "LikertScaleQuestions", and "OpenEndedQuestions" as sub-classes of "SurveyQuestions", and "LikertScaleAnswers", and "OpenEndedAnswers" as sub-classes of "SurveyAnswers". For all the survey questions, there is a question and an answer, where both of them are considered as distinct classes. Open-ended questions have unpredictable answers.

4.1.1.1 The "LikertScaleQuestions" Class

Likert questions are the questions for which the answers are from the preset list of answers. For the Likert questions, the questions can be either of type "Never-Always-4"

[1]https://www.surveymonkey.com/mp/survey-question-types/

or "Never-Always-5" or "Never-Always-7" or "Never-Always-*", of which Never-Always-5 has been used by Hepting [29]. The survey ontology consists of "LikertScaleQuestions" class for likert questions.

In a Never-Always-5 type of survey, 5 degrees of agreement can be used. It is challenging to decide whether the range for the degrees would be 0 to 4, 1 to 5, 10 to 50, or something else. The degree of agreements used here are as shown in Table 4.1. In addition to these degrees, this ontology also considers the instances where the question wouldn't be answered. In this case, the field corresponding to the answer is left blank.

Number	Agreement
1	Never
2	[Rarely]
3	[Sometimes]
4	[Often]
5	Always

Table 4.1: Degrees of Agreement for Likert-scale Questions

The "LikertScaleQuestions" class of survey ontology has a sub-class, "LikertScale5", which represents the likert scale with the degrees of agreement from Table 4.1. The "Equivalent To" (from the description view of the class tab) can be used to describe the "LikertScale5" class in terms of a range containing degrees of agreement as shown in Figure 4.3. Each of these degrees are treated as an individual (section 4.1.4) of the class.

4.1.2 Object Properties for Survey Ontology

Using the method of naming and building object properties described in section 3.2.2, some of the primary object properties are described for the survey ontology as shown in Figure 4.4 where as Table 4.2 shows the list of the object properties along with their domain and range classes. Table 4.3 consists of the parent nodes and inverse properties

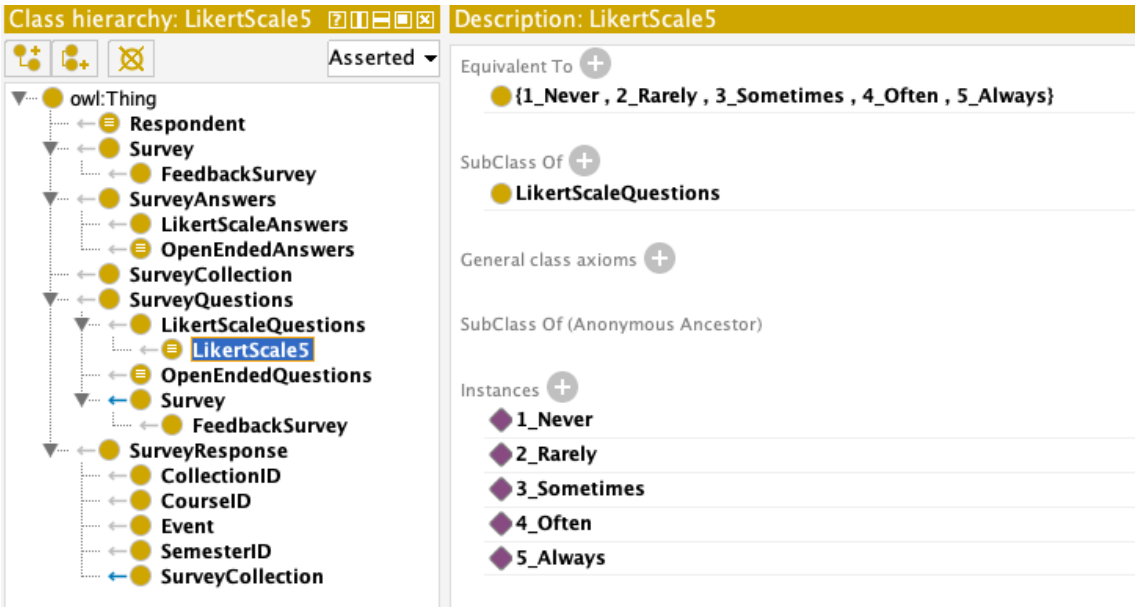

Figure 4.3: "LikertScale5" class in Survey Ontology

of the object properties. The three parent nodes involved in object property hierarchy are "owl:TopObjectProperty", "hasContent", and "isContentOf".

Object Property	Domain	Range
hasSurveyQuestion	Survey	SurveyQuestions
hasSurveyAnswer	Survey	SurveyAnswers
hasOEAnswer_Change	OpenEndedQuestions	OpenEndedAnswers
hasOEAnswer_LikeBest	OpenEndedQuestions	OpenEndedAnswers
hasOEAnswer_Strengths	OpenEndedQuestions	OpenEndedAnswers
hasLikertScaleAnswer	LikertScaleQuestions	LikertScale5
hasEvent	SurveyResponse	Event
hasCourseID	SurveyResponse	CourseID
hasSemesterID	SurveyResponse	SemesterID
hasCollectionID	SurveyResponse	CollectionID
isCollectionOf	SurveyCollection	SurveyResponse
respondedToSurvey	Respondent	Survey
isTypeOf	FeedbackSurvey	Survey

Table 4.2: Object Properties and Their Domain and Range

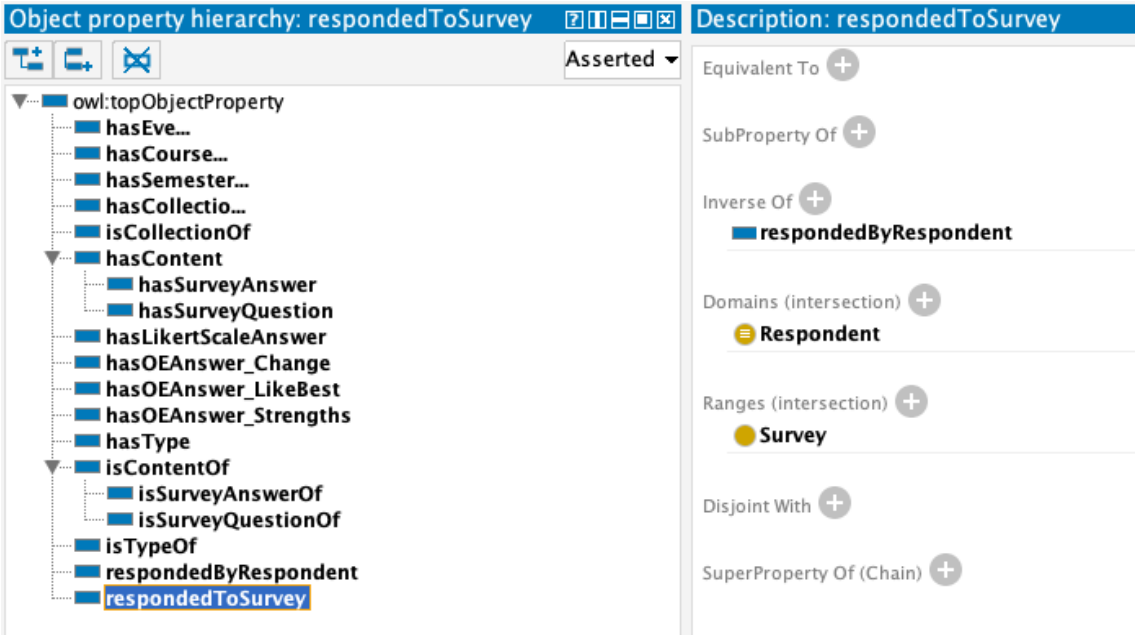

Figure 4.4: Object Property Hierarchies, Domain and Range for Classes

Object Property	Parent Node	Inverse Property
hasContent	owl:TopObjectProperty	isContentOf
isContentOf	owl:TopObjectProperty	hasContent
hasType	owl:TopObjectProperty	isTypeOf
isTypeOf	owl:TopObjectProperty	hasType
hasLikertScaleAnswer	owl:TopObjectProperty	
hasOEAnswer_Change	owl:TopObjectProperty	
hasOEAnswer_LikeBest	owl:TopObjectProperty	
hasOEAnswer_Strengths	owl:TopObjectProperty	
hasEvent	owl:TopObjectProperty	
hasCourseID	owl:TopObjectProperty	
hasSemesterID	owl:TopObjectProperty	
hasCollectionID	owl:TopObjectProperty	
isCollectionOf	owl:TopObjectProperty	
respondedByRespondent	owl:TopObjectProperty	respondedToSurvey
respondedToSurvey	owl:TopObjectProperty	respondedByRespondent
hasSurveyQuestion	hasContent	isSurveyQuestionOf
hasSurveyAnswer	hasContent	isSurveyAnswerOf
isSurveyQuestionOf	isContentOf	hasSurveyQuestion
isSurveyAnswerOf	isContentOf	hasSurveyAnswer

Table 4.3: Object Properties with Their Parent Nodes and Inverse Properties

4.1.3 Data Properties for Survey Ontology

Data properties for survey ontology have been described following the method described in section 3.2.3 which are as shown in Figure 4.5.

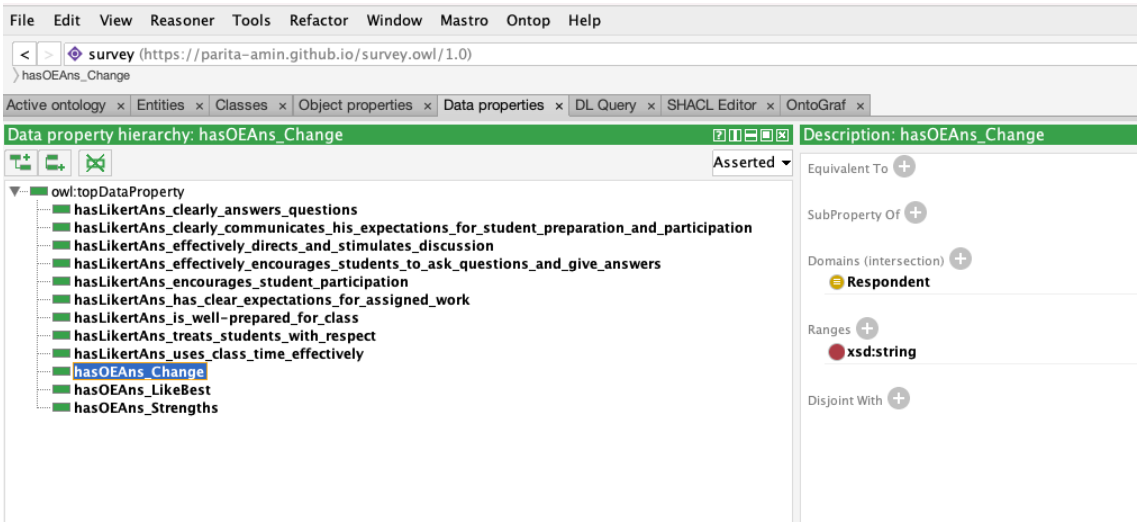

Figure 4.5: Data Property Hierarchies, Domain and Range for Classes

The parent node of all the data properties is "owl:topDataProperty" and the domain and range for all the data properties are listed in Table 4.4. The properties for likert answers restrict the datatype to positive integer and to string for open-ended answers.

4.1.4 Individuals

While dealing with ontologies, an instance of class is termed as an "Individual". In protégé, an individual can be described as an instance of class using the Description view of the Class tab. For example, Figure 4.6 shows instances of "Survey" class.

A new individual can be described using the Individuals view of the Entities tab where as the property assertions view can be used to describe the property restrictions. Table 4.5 shows a list of classes and individuals described in survey ontology.

Data Property	Domain	Range
hasLikertAns_is_well-prepared_for_class	Respondent	xsd:positiveInteger
hasLikertAns_clearly_communicates_his_expectations_for_student_preparation_and_participation	Respondent	xsd:positiveInteger
hasLikertAns_uses_class_time_effectively	Respondent	xsd:positiveInteger
hasLikertAns_has_clear_expectations_for_assigned_work	Respondent	xsd:positiveInteger
hasLikertAns_encourages_student_participation	Respondent	xsd:positiveInteger
hasLikertAns_clearly_answers_questions	Respondent	xsd:positiveInteger
hasLikertAns_treats_students_with_respect	Respondent	xsd:positiveInteger
hasLikertAns_effectively_directs_and_stimulates_discussion	Respondent	xsd:positiveInteger
hasLikertAns_effectively_encourages_students_to_ask_questions_and_give_answers	Respondent	xsd:positiveInteger
hasOEAnswer_Change	Respondent	xsd:string
hasOEAnswer_LikeBest	Respondent	xsd:string
hasOEAnswer_Strengths	Respondent	xsd:string

Table 4.4: Data Properties and Their Domain and Range

Class	Individuals
Respondent	1, 2, 3, ... , 59
Survey	CS280/202210, CS499+900/202210, CS730/202210
SurveyCollection	Collection1, Collection2
Event	FinalExam, Midterm
CollectionID	CS280/202210/Collection1
SemesterID	202210, 202220, 202230
CourseID	CS280, CS499+900, CS730
OpenEndedQuestions	OEQuestion_LikeBest, OEQuestion_Change, OEQuestion_Strengths
OpenEndedAnswers	OEAns_LikeBest, OEAns_Change, OEAns_Strengths
LikertScale5	1_Never, 2_Rarely, 3_Sometimes, 4_Often, 5_Always

Table 4.5: Classes and their Individuals

4.1.5 Final Class Hierarchy for Survey Ontology

The final class hierarchy of the OWL ontology in protégé, after using the HermiT 1.4.3.456 reasoner, would be as shown in Figure 4.7.

The blue ("Survey" and "SurveyCollection" class) back arrow in front of the yellow

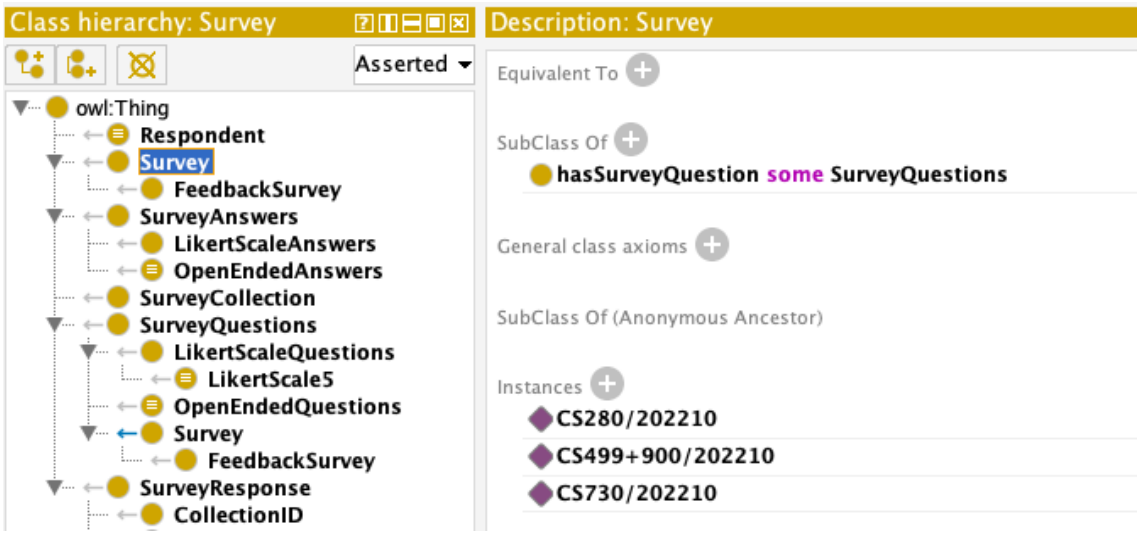

Figure 4.6: Individuals for Survey Class

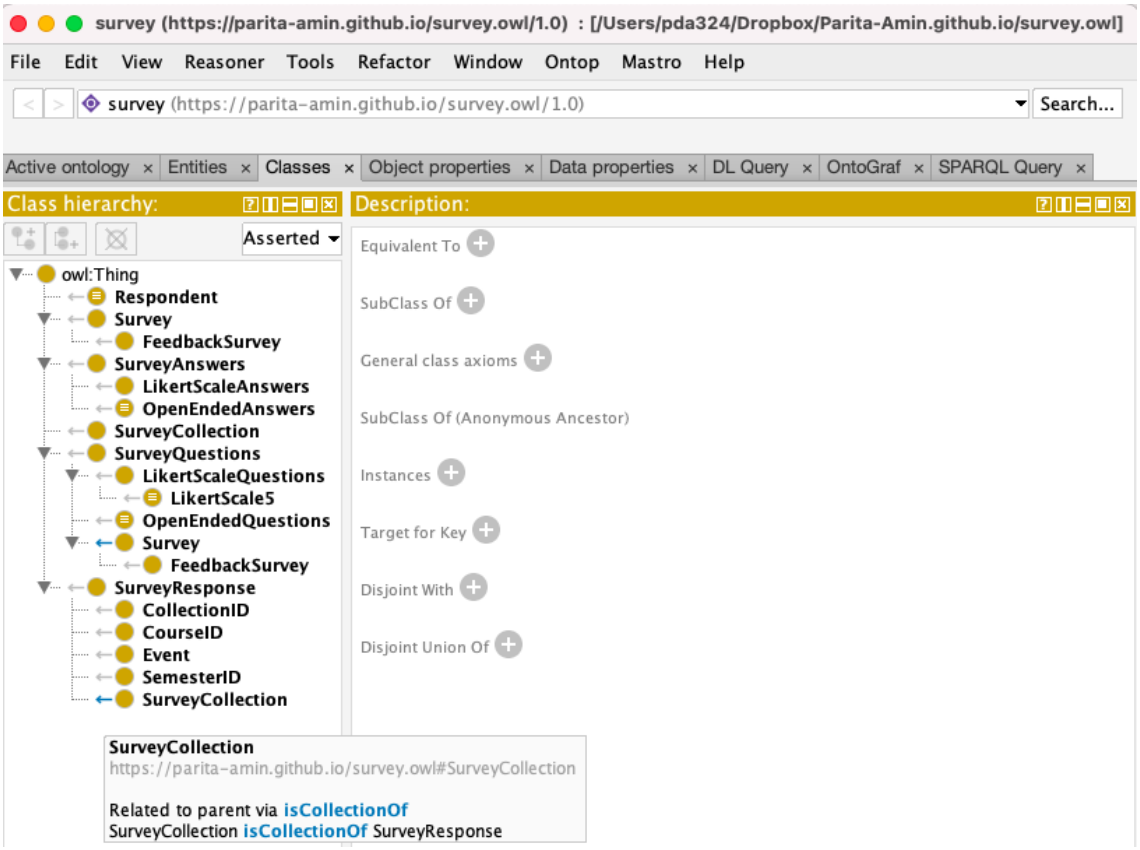

Figure 4.7: Class Hierarchy for the Final Version of Survey Ontology

solid dot, which indicates class, show parent/child relationship other than that of SubClassOf. This feature can be enabled and disabled by selecting "Display relationship in class hierarchy" option from the "View" menu of protégé. Figure 4.7 shows the relation indicated by the blue back arrow.

4.1.6 Using SHACL for Data Validation

SHACL4P is a protégé plugin for SHACL, which uses "shapes" to describe constraints. SHACL can be used to validate the manually entered data which are stored in spreadsheets. For feedback survey, the likert answers can only have values 1, 2, 3, 4, 5, or No Answer (blank cell). To restrict the values of data properties corresponding to likert questions in Figure 4.5, SHACL4P has been used.

Protégé offers a view for SHACL editor as well as SHACL constraint violations. For successful working of SHACL4P, a reasoner should be enabled. The SHACL editor provides three buttons, namely, "Open", "Save" , and "Validate". After opening the shapes file (Open button), changes can be made and saved (Save button). Where as the Validate button allows users to validate the data. Figure 4.8 shows a sample "SurveyShapes.txt" file opened in SHACL editor of protégé.

On clicking the Validate button, the summary of violated constraints is displayed on the SHACL constraint violations view in protégé. It contains the details like "Severity", "SourceShape", "Message", "FocusNode", "Path", and "Value" as shown in Figure 4.9.

4.1.7 Querying Survey Ontology in Protégé

The ontology vocabularies can be queried using query languages like DL query and SPARQL. A DL query is used to such as 'and' (corresponding to set intersection) and 'some'.[2]. SPARQL query is used to express queries and optional graph patterns along

[2]https://ontology101tutorial.readthedocs.io/en/latest/EXERCISE_BasicDL_Queries.html

SHACL editor:

Open Save Validate

```
@prefix rdf:   <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix sh:    <http://www.w3.org/ns/shacl#> .
@prefix xsd:   <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs:  <http://www.w3.org/2000/01/rdf-schema#> .
@prefix survey: <https://parita-amin.github.io/survey.owl#> .
@prefix owl:   <http://www.w3.org/2002/07/owl#> .

survey:RespondentShape
    a sh:NodeShape ;
    sh:targetClass survey:Respondent ; # Applies to all Respondents

            sh:property [
        sh:path survey:hasLikertAns_clearly_answers_questions ;  # constrains the values of survey:hasLikertAns_clearly_answers_questions
        sh:datatype xsd:positiveInteger ;
        sh:minInclusive  1 ;
        sh:maxInclusive 5 ;
        sh:severity sh:Warning ;
    ] ;



    sh:closed false ;
    sh:ignoredProperties ( rdf:type owl:topDataProperty owl:topObjectProperty ) ;
    .
###### add SHACL vocabulary ######
```

Figure 4.8: Data Validation using SHACL Editor

SHACL constraint violations: 1593

Severity	SourceShape	Message	FocusNode	Path	Value
http://www.w3.org/ns/shacl#Warning	bb5a1ac8f...	Cannot compare with 1	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_clearly_answers_questions	1
http://www.w3.org/ns/shacl#Warning	bb5a1ac8f...	Cannot compare with 5	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_clearly_answers_questions	1
http://www.w3.org/ns/shacl#Warning	bb5a1ac8f...	Value must be a valid literal of type positiveInteger	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_clearly_answers_questions	1
http://www.w3.org/ns/shacl#Warning	22f8124d...	Cannot compare with 1	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_clearly_communicates_his_...	1
http://www.w3.org/ns/shacl#Warning	22f8124d...	Cannot compare with 5	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_clearly_communicates_his_...	1
http://www.w3.org/ns/shacl#Warning	22f8124d...	Value must be a valid literal of type positiveInteger	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_clearly_communicates_his_...	1
http://www.w3.org/ns/shacl#Warning	c1818e90...	Cannot compare with 1	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_effectively_directs_and_sti...	1
http://www.w3.org/ns/shacl#Warning	c1818e90...	Cannot compare with 5	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_effectively_directs_and_sti...	1
http://www.w3.org/ns/shacl#Warning	c1818e90...	Value must be a valid literal of type positiveInteger	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_effectively_directs_and_sti...	1
http://www.w3.org/ns/shacl#Warning	92359815...	Cannot compare with 1	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_effectively_encourages_stu...	1
http://www.w3.org/ns/shacl#Warning	92359815...	Cannot compare with 5	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_effectively_encourages_stu...	1
http://www.w3.org/ns/shacl#Warning	92359815...	Value must be a valid literal of type positiveInteger	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_effectively_encourages_stu...	1
http://www.w3.org/ns/shacl#Warning	127be816...	Cannot compare with 1	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_encourages_student_partic...	3
http://www.w3.org/ns/shacl#Warning	127be816...	Cannot compare with 5	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_encourages_student_partic...	3
http://www.w3.org/ns/shacl#Warning	127be816...	Value must be a valid literal of type positiveInteger	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_encourages_student_partic...	3
http://www.w3.org/ns/shacl#Warning	088f2ba8...	Cannot compare with 1	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_has_clear_expectations_fo...	1
http://www.w3.org/ns/shacl#Warning	088f2ba8...	Cannot compare with 5	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_has_clear_expectations_fo...	1
http://www.w3.org/ns/shacl#Warning	088f2ba8...	Value must be a valid literal of type positiveInteger	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_has_clear_expectations_fo...	1
http://www.w3.org/ns/shacl#Warning	51c5d0d3...	Cannot compare with 1	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_is_well-prepared_for_class	2
http://www.w3.org/ns/shacl#Warning	51c5d0d3...	Cannot compare with 5	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_is_well-prepared_for_class	2
http://www.w3.org/ns/shacl#Warning	51c5d0d3...	Value must be a valid literal of type positiveInteger	http://127.0....	https://parita-amin.github.io/survey.owl#hasLikertAns_is_well-prepared_for_class	2

Figure 4.9: SHACL Constraint Violations

with their conjunctions and disjunctions as well as aggregation, subqueries, negation, etc.[3]

Protégé offers a plugin for DL as well as SPARQL queries.

4.1.7.1 DL Query

"The DL Query tab allows users to quickly test definitions of classes to see that they subsume the appropriate subclasses. Or check for class membership of arbitrary descriptions without having to create named class placeholders."[4] The DL Query tab can be opened using Tabs option under Windows menu. It is important to configure a reasoner to successfully express a DL query. Figure 4.10 shows expression of a sample DL query on the survey ontology in protégé.

4.1.7.2 SPARQL Query

4.2 From Spreadsheets to Linked Open Data For Survey Ontology

Primarily, OpenRefine[5] was used to convert the data stored in spreadsheets to Turtle and RDF files.

[3]https://www.w3.org/TR/sparql11-query/
[4]https://protegewiki.stanford.edu/wiki/DL_Query
[5]https://openrefine.org/

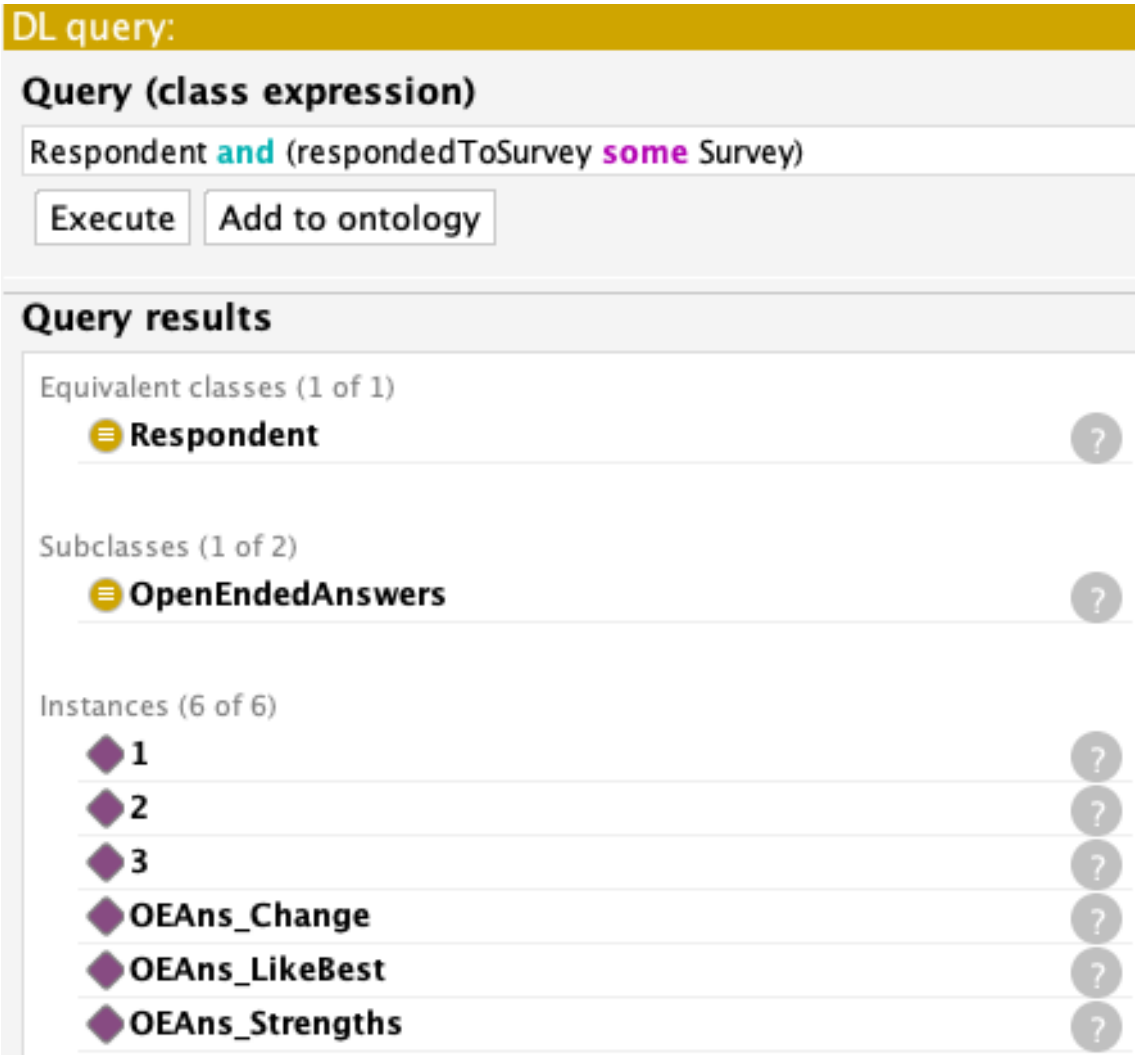

Figure 4.10: DL Query on Survey Ontology

SPARQL query:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
SELECT ?subject ?object
        WHERE { ?subject rdfs:subClassOf ?object }
```

subject	object
CollectionID	SurveyResponse
Survey	hasSurveyQuestion some SurveyQuestions
LikertScaleAnswers	SurveyAnswers
OpenEndedAnswers	SurveyAnswers
LikertScale5	LikertScaleQuestions
SemesterID	SurveyResponse
CourseID	SurveyResponse
OpenEndedQuestions	SurveyQuestions
Event	SurveyResponse
LikertScaleQuestions	SurveyQuestions
FeedbackSurvey	Survey
SurveyCollection	isCollectionOf some SurveyResponse

Figure 4.11: SPARQL Query on Survey Ontology

Snap SPARQL Query:

```
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT (STR(?lab) AS ?Class) ?Individuals WHERE {
            ?Individuals rdf:type owl:NamedIndividual .
            OPTIONAL {?Individuals rdf:type ?lab}
}
ORDER BY ?Class
```

Execute

?Class	?Individuals
https://parita-amin.github.io/survey.owl#CourseID	<https://parita-amin.github.io/survey.owl#CS499+900>
https://parita-amin.github.io/survey.owl#Event	<https://parita-amin.github.io/survey.owl#FinalExam>
https://parita-amin.github.io/survey.owl#Event	<https://parita-amin.github.io/survey.owl#Midterm>
https://parita-amin.github.io/survey.owl#LikertScale5	<https://parita-amin.github.io/survey.owl#1_Never>
https://parita-amin.github.io/survey.owl#LikertScale5	<https://parita-amin.github.io/survey.owl#2_Rarely>
https://parita-amin.github.io/survey.owl#LikertScale5	<https://parita-amin.github.io/survey.owl#3_Sometimes>
https://parita-amin.github.io/survey.owl#LikertScale5	<https://parita-amin.github.io/survey.owl#4_Often>
https://parita-amin.github.io/survey.owl#LikertScale5	<https://parita-amin.github.io/survey.owl#5_Always>

84 results

Figure 4.12: Snap SPARQL Query on Survey Ontology

4.3 Licensing Survey Vocabulary

4.4 Publishing Linked Open Vocabulary for Survey

4.5 Visualization Tools for OWL Ontology

The next step is visualization of the built OWL ontology using various visualization tools. Protégé supports plug-ins for some visualization tools like OntoGraf and OWLViz whereas tools like WebVOWL, RelFinder and OWLGrEd are available for use as web-based tools. This section focuses on the two tools, OntoGraf and WebVOWL, used for the visualization of the survey ontology in this research.

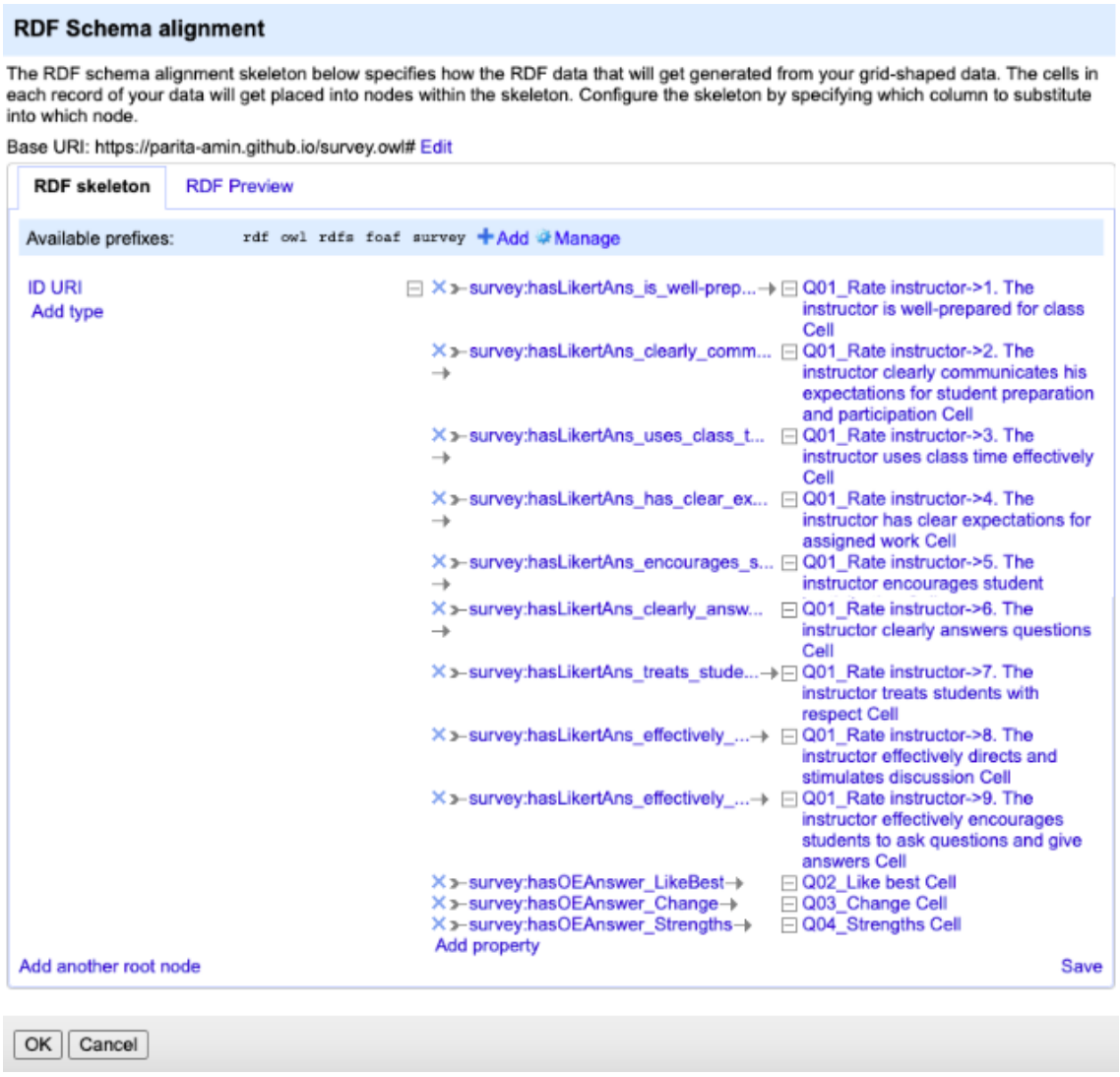

Figure 4.13: RDF Skeleton for CSV using Survey Ontology

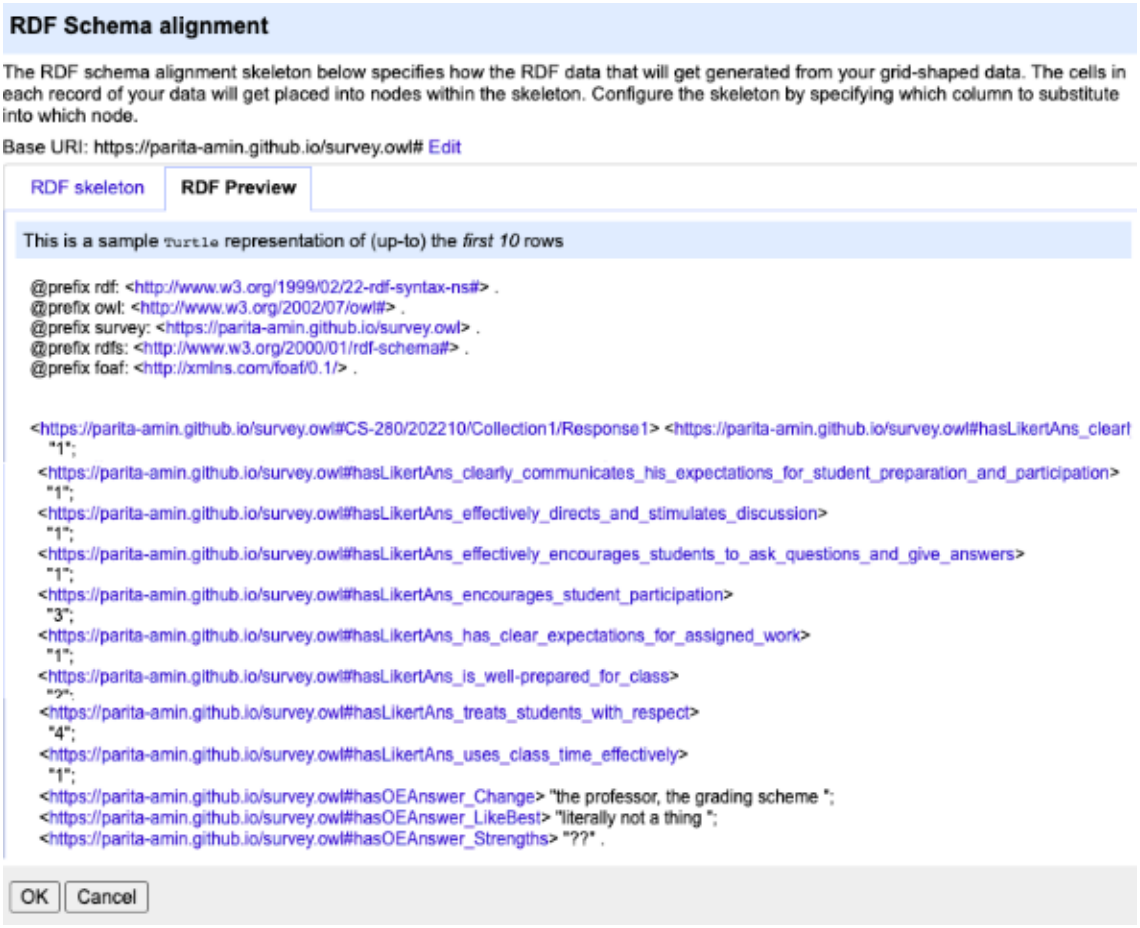

Figure 4.14: RDF Preview for CSV using Survey Ontology

4.5.1 OntoGraf

Falconer [25] introduced OntoGraf as a protégé plug-in for the visualization of the OWL ontologies. This tool helps the users to represent the ontologies, built using protégé, by repetitively allowing and restricting the desired classes. OntoGraf offers various graph layout options for visualization like Grid Layout[6], Spring Layout, and Tree Layout[7]. Antoniazzi and Viola [3] stated that the "individuals of a class can be visualized in its tooltip, but this is uncomfortable when dealing with a high number of assertional statements". The graphs generated using OntoGraf can be exported as image files of various formats such as Portable Network Graphics (PNG), Joint Photographic Experts Group (JPEG), Graphics Interchange Format (GIF) and as a dot file[8].

Figure 4.15 shows the graph generated using OntoGraf for the survey ontology. To start with the graph formation, the classes "Thing", "Domain_entity", "Independent_entity" and "Value" were selected followed by expansion of "Independent_entity" and "Value" classes by double-clicking them. This expansion adds all the classes that are connected to the classes. The classes connected includes both the subclasses as well as the ones linked through the object properties. Solid blue lines are used to denote the relationships between the classes and their subclasses whereas dashed lines are used to represent the object properties.

4.5.2 WebVOWL

WebVOWL [33], the Web-based Visual Web Ontology Language (VOWL), one of the variety of forms in which the VOWL visualization tool is available, represents the "ontologies graphically using a force-directed graph layout" [3]. VOWL follows some ground rules for the representation of OWL ontologies which are:

[6]classes are arranged in a grid alphabetically
[7]both horizontal and vertical
[8]a Microsoft Word Template with the details of default settings

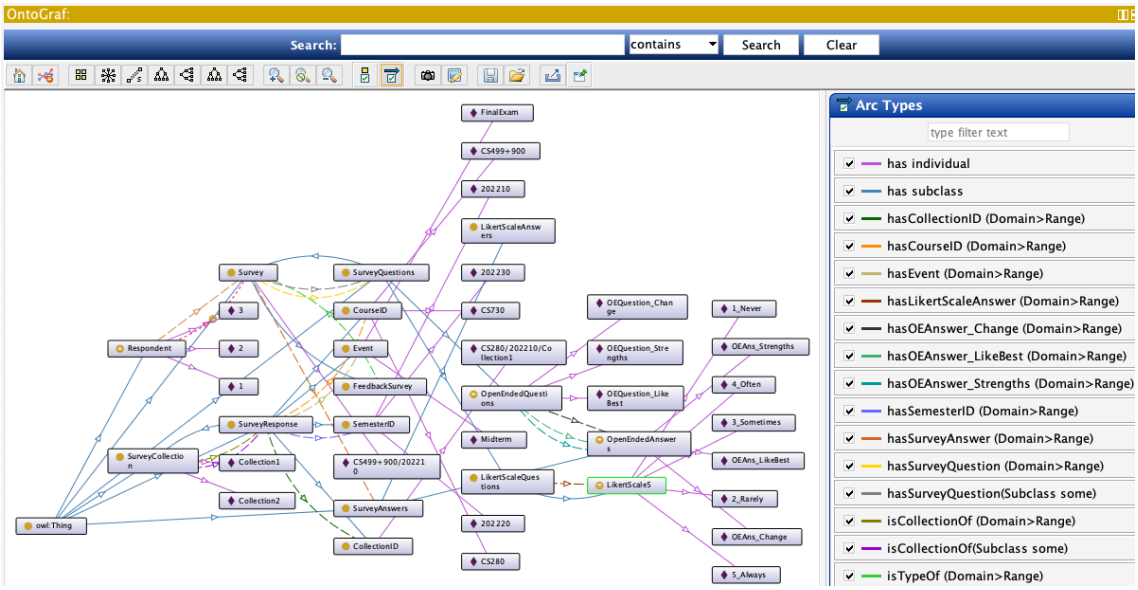

Figure 4.15: OntoGraf for Survey Ontology

1. Circles are used to denote classes. Also, different types are assigned different colors. For instance, OWL classes are denoted using "light blue circles".

2. Black solid lines are used to represent OWL object and datatype properties. The object properties use light blue labels where as the datatype properties are labelled in green.

3. The relationships between the classes and their subclasses are represented using dashed lines.

Figure 4.16 shows the graphical representation of the survey ontology using WebVOWL. The figure focuses on only some of the class-subclass relationships and some of the object properties. The metadata and the statistics of the node (circle) or the edge (line) can be retrieved by clicked on them. The entities (classes) and their relationships (object and datatype properties) can be presented or restricted using filters. The VOWL graph can be exported as an Scalable Vector Graphics (SVG) image file or a JSON code file.

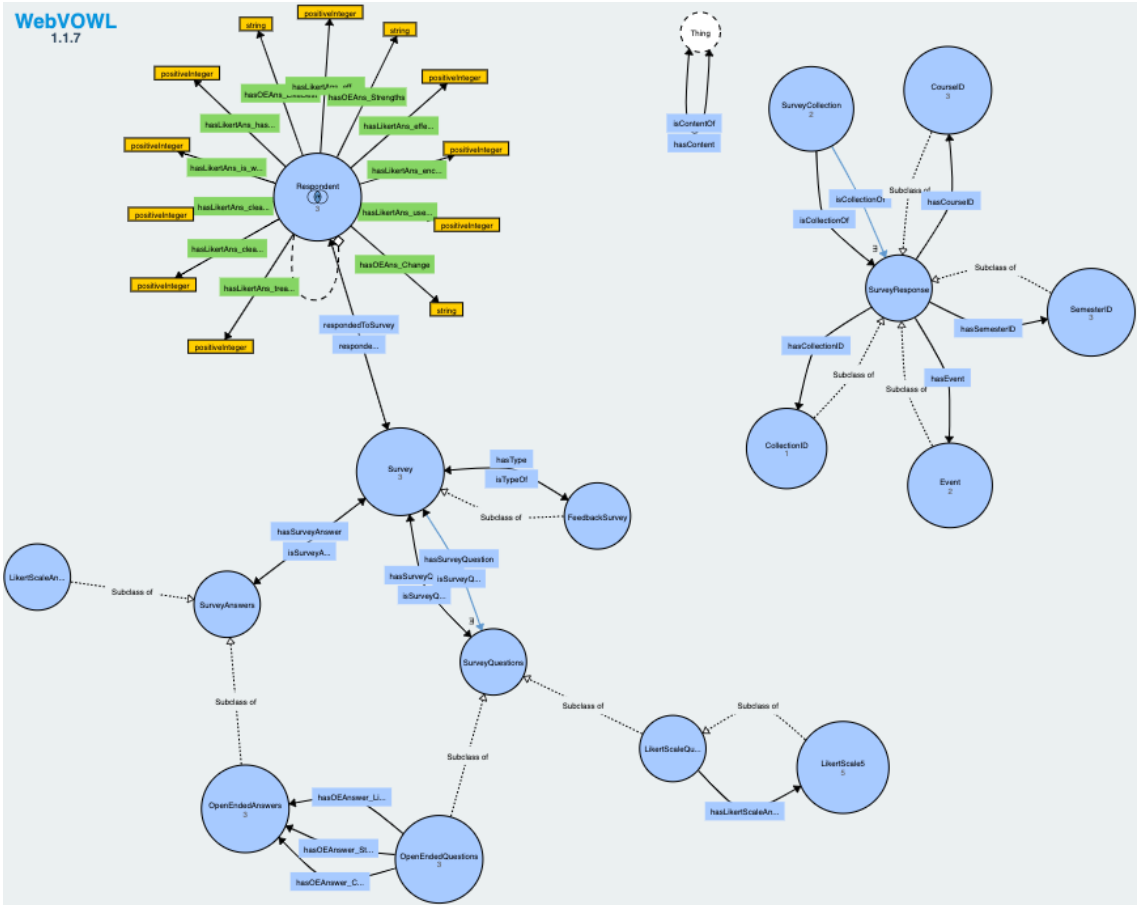

Figure 4.16: WebVOWL for Survey Ontology

Chapter 5

Conclusion and Future Work

This Chapter focuses on the prime offerings of this research work where one of the sections includes the conclusions drawn from the work done and the other discusses future work that can be done to succeed in this research.

5.1 Conclusion

This thesis work describes a new attempt on developing a linked open data instrument for feedback surveys. It focuses on the issues encountered and the user experience of various tools and references used for the modelling of the instrument and for data visualization. The thesis revolves around some fundamental topics like Semantic Web, Linked Open Data, RDF, and OWL ontologies. It also describes how and why Protégé has been used over other ontology development tools to describe survey vocabularies. The thesis can be considered as a guide for the researchers who are interested in exploring this lesser known aspect of semantic web.

The thesis includes a detailed background study on the fundamental topics, the usability of protégé, and discussion on some tools for linked open data visualizations. Chapter 4 describes the steps involved in building the survey ontology using protégé, various parts and facilities offered by protégé, and the visual representation of the

ontology toward the end.

5.2 Future Work

This thesis also offers a number of future development options like building a real-time linked open data application for feedback surveys that follows the linked data life cycle. Web Scraping (or Web Extraction) is a method of extracting desired data from the resources on the WWW into a separate file for data retrieval and data analysis. Data Extraction is one of the steps in the linked data life cycle that can be explored in future research to facilitate use of data that has already been published to the web with HTML tables.

Versioning of the survey ontology is a topic of interest. The ontology vocabularies can be reused for appropriate applications but they may need to be updated over time. Hence, ontology versioning and version management of the survey instrument for this feedback survey, as well as for other types of surveys is an aspect of this research that can be explored.

Developing a data entry form for the feedback survey instrument that can be defined, in part, directly from the survey ontology would facilitate recording of survey responses collected offline.

Bibliography

[1] Sareh Aghaei, Mohammad Ali Nematbakhsh, and Hadi Khosravi Farsani. Evolution of the World Wide Web: From WEB 1.0 TO WEB 4.0. *International Journal of Web and Semantic Technology*, 3(1):1–10, 2012.

[2] Emhimed Alatrish. Comparison Some of Ontology Editors. *Management Information Systems*, 8(2):18–24, 2013.

[3] Francesco Antoniazzi and Fabio Viola. RDF Graph Visualization Tools: A Survey. In *Proceedings of the 23rd Conference of Open Innovations Association (FRUCT)*, pages 25–36, 2018.

[4] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. DBpedia: A Nucleus for a Web of Open Data. In *Proceedings of the 6th International Conference on the Semantic Web and 2nd Asian Conference on Asian Semantic Web Conference*, pages 722–735. Springer-Verlag, Berlin, Heidelberg, 2007.

[5] Florian Bauer and Martin Kaltenböck. Linked Open Data: The Essentials - A Quick Start Guide for Decision Makers. *Edition Mono/Monochrom*, Vienna, 2011.

[6] Dave Beckett and Brian McBride. RDF/XML Syntax Specification (Revised). *W3C Recommendation*, 2004. https://www.w3.org/TR/REC-rdf-syntax/ , accessed August 14, 2019.

[7] David Beckett. RDF 1.1 N-Triples. *W3C Recommendation*, 2014. https://www.w3.org/TR/2014/REC-n-triples-20140225/ , accessed October 29, 2019.

[8] Tim Berners-Lee. Linked Data. *W3C Recommendation*, 1996. https://www.w3.org/DesignIssues/LinkedData , accessed January 23, 2019.

[9] Tim Berners-Lee. Universal Resource Identifiers in WWW: A Unifying Syntax for the Expression of Names and Addresses of Objects on the Network as used in the World-Wide Web. *RFC 1630*, June 1998. https://www.rfc-editor.org/info/rfc1630 , accessed September 24, 2020.

[10] Tim Berners-Lee, Robert Cailliau, Ari Luotonen, Henrik Frystyk Nielsen, and Arthur Secret. The World Wide Web. *Communications of the ACM*, 37(8):76–83, 1994.

[11] Tim Berners-Lee, Roy Fielding, and Larry Masinter. Uniform Resource Identifiers (URI): Generic Syntax. *Internet Engineering Task Force, RFC 2396*, August 1998. https://www.rfc-editor.org/info/rfc2396 , accessed October 01, 2020.

[12] Tim Berners-Lee, Roy Fielding, and Larry Masinter. Uniform Resource Identifiers (URI): Generic Syntax. *RFC 3986*, January 2005. https://www.rfc-editor.org/info/rfc3986 , accessed October 02, 2020.

[13] Tim Berners-Lee and Mark Fischetti. *Weaving the Web: The Original Design and Ultimate Destiny of the World Wide Web by Its Inventor*. Harper San Francisco, 1999.

[14] Christian Bizer. The Emerging Web of Linked Data. *IEEE Intelligent Systems*, 24(5):87–92, 2009.

[15] Christian Bizer, Tom Heath, and Tim Berners-Lee. Linked Data - The Story So Far. *International Journal of Semantic Web Information Systems*, 5:1–22, 2009.

[16] Christian Bizer, Tom Heath, and Tim Berners-Lee. Linked Data: The Story So Far. In *Semantic Services, Interoperability and Web Applications: Emerging Concepts*, pages 205–227. IGI global, USA, 2011.

[17] Christian Bizer, Tom Heath, Kingsley Idehen, and Tim Berners-Lee. Linked Data on the Web (LDOW2008). In *Proceedings of the 17th International Conference on World Wide Web*, pages 1265–1266. ACM, 2008.

[18] Christian Bizer, Jens Lehmann, Georgi Kobilarov, Sören Auer, Christian Becker, Richard Cyganiak, and Sebastian Hellmann. DBpedia-A Crystallization Point for the Web of Data. *Web Semantics: Science, Services and Agents on the World Wide Web*, 7(3):154–165, 2009.

[19] Nupur Choudhury. World Wide Web and Its Journey from Web 1.0 to Web 4.0. *International Journal of Computer Science and Information Technologies*, 5(6):8096–8100, 2014.

[20] Richard Cyganiak, Andreas Harth, and Aidan Hogan. N-Quads: Extending N-Triples with Context. *W3C Recommendation*, pages 1–41, 2008.

[21] Richard Cyganiak, David Wood, Markus Lanthaler, Graham Klyne, Jeremy J Carroll, and Brian McBride. RDF 1.1 Concepts and Abstract Syntax. *W3C Recommendation*, 25(02):1–22, January 2014. http://hdl.handle.net/10421/2427 , accessed January 05, 2020.

[22] Fajar J. Ekaputra and Xiashuo Lin. SHACL4P: SHACL constraints validation within Protégé ontology editor. In *2016 International Conference on Data and Software Engineering (ICoDSE)*, pages 1–6, Oct 2016.

[23] Ivan Ermilov, Sören Auer, and Claus Stadler. CSV2RDF: User-Driven CSV to RDF Mass Conversion Framework. In *Proceedings of the International Symposium on Applied Electromagnetics and Mechanics*, volume 13, pages 4–6. ACM, 2013.

[24] Fredo Erxleben, Michael Günther, Markus Krötzsch, Julian Mendez, and Denny Vrandečić. Introducing Wikidata to the Linked Data Web. In *Proceedings of the International Semantic Web Conference*, pages 50–65. Springer, 2014.

[25] Sean Falconer. Ontograf Protégé Plugin. *Available at: http://protegewiki.stanford.edu/wiki/OntoGraf*, 2010.

[26] Thomas R Gruber. Toward Principles for the Design of Ontologies used for Knowledge Sharing? *International Journal of Human-Computer Studies*, 43(5):907–928, 1995.

[27] Lushan Han, Tim Finin, Cynthia Parr, Joel Sachs, and Anupam Joshi. RDF123: From Spreadsheets to RDF. In *Proceedings of the International Semantic Web Conference*, pages 451–466. Springer, 2008.

[28] Tom Heath and Christian Bizer. Linked Data: Evolving the Web into a Global Data Space. *Synthesis Lectures on the Semantic Web: Theory and Technology*, 1(1):1–136, 2011.

[29] Daryl Hepting. Student Feedback Instrument. *Available at: http://www2.cs.uregina.ca/~hepting/assets/teaching/pdf/feedback-instrument.pdf*, 2020.

[30] Matthew Horridge, Simon Jupp, Georgina Moulton, Alan Rector, Robert Stevens, and Chris Wroe. A Practical Guide to Building OWL Ontologies using Protégé 4 and CO-ODE Tools Edition 1.3. *The University of Manchester*, March 2011.

[31] Elin K Jacob. Ontologies and the Semantic Web. *Bulletin of the American Society for Information Science and Technology*, 29(4):19–19, 2003.

[32] Andreas Langegger and Wolfram Wob. XLWrap–Querying and Integrating Arbitrary Spreadsheets with SPARQL. In *Proceedings of the International Semantic Web Conference*, pages 359–374. Springer, 2009.

[33] Steffen Lohmann, Vincent Link, Eduard Marbach, and Stefan Negru. WebVOWL: Web-based Visualization of Ontologies. In *International Conference on Knowledge Engineering and Knowledge Management*, pages 154–158. Springer, 2014.

[34] Frank Manola, Eric Miller, and Brian McBride. RDF Primer. *W3C Recommendation*, 10(1):1–6, 2004. https://www.w3.org/TR/2004/REC-rdf-primer-20040210/ , accessed December 2017.

[35] Brian McBride. The Resource Description Framework (RDF) and Its Vocabulary Description Language RDFS. In *Handbook on Ontologies*, pages 51–65. Springer, Berlin, 2004.

[36] Axel-Cyrille Ngonga Ngomo, Sören Auer, Jens Lehmann, and Amrapali Zaveri. Introduction to Linked Data and Its Life Cycle on the Web. In *Proceedings of Reasoning Web 2014: Reasoning on the Web in the Big Data Era*, pages 1–99, Cham. Springer International, September 2014.

[37] Jakob Nielsen. Usability Engineering. *Morgan Kaufmann Publichers Inc., San Franscisco, USA*, pages 23–48, 1994.

[38] Irene Petrou, Marios Meimaris, and George Papastefanatos. Towards a Methodology for Publishing Linked Open Statistical Data. *JeDEM - eJournal of eDemocracy and Open Government*, 6(1):97–105, Nov 2014.

[39] Whitney Quesenbery and Whitney Interactive Design. Dimensions of Usability: Defining the Conversation, Driving the Process. In *UPA 2003 Conference*, pages 23–27, 2003.

[40] Denny Vrandečić and Markus Krötzsch. Wikidata: A Free Collaborative Knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014.